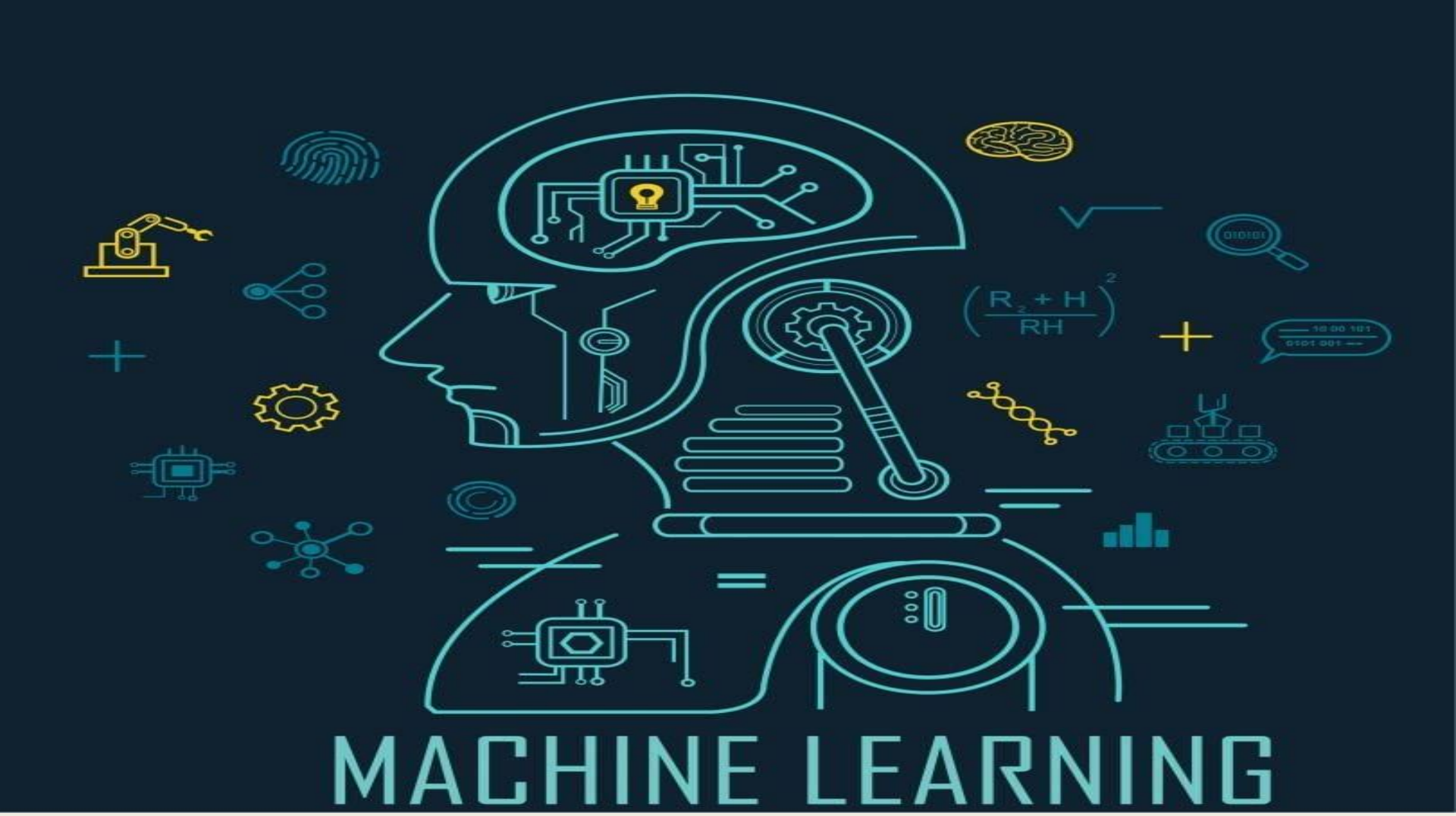


UNIT-1

Machine Learning

TEXT BOOKS:

- **Stephen Marsland, —Machine Learning – An Algorithmic Perspective‖, Second Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2014.**

- **2. Tom M Mitchell, —Machine Learning‖, First Edition, McGraw Hill Education, 2013.**

REFERENCES:

- **Peter Flach, —Machine Learning: The Art and Science of Algorithms that Make Sense of Data‖, First Edition, Cambridge University Press, 2012.**

- **Jason Bell, —Machine learning – Hands on for Developers and Technical Professionals‖, First Edition, Wiley, 2014**

- **3. Ethem Alpaydin, —Introduction to Machine Learning 3e (Adaptive Computation and Machine Learning Series), Third Edition, MIT Press, 2014**

Machine Learning

UNIT I: INTRODUCTION:

- Learning(Book-1) – Types of Machine Learning – Supervised Learning – The Brain and the Neuron – Design a Learning System(Book-2) – Perspectives and Issues in Machine Learning – Concept Learning Task – Concept Learning as Search – Finding a Maximally Specific Hypothesis – Version Spaces and the Candidate Elimination Algorithm – Linear Discriminants: (Book-1) – Perceptron – Linear Separability – Linear Regression.

Learning

- Learn from data (TB)
- It gives us flexibility

Defn : A computer program is said to ‘Learn’ from experience ‘E’ wrt class of tasks ‘T and performance measure ‘P’ if the performance at tasks improves with ‘E’.

Examples

	Checkers game	Handwriting recognition	Robot driving learning
T	Playing checkers	Recognizing and classifying handwritten words in images	Driving on public 4 lane highway using sensors
P	Performance measure	% of words correctly classified	Avg. distance travelled before error
E	Playing practice games	Database of handwritten words and classification	Sequence of images and steering commands

Machine Learning

- It makes computers modify or adapt to their actions
- It merges ideas from :
- Neuroscience \biology\stats\maths\physics

Data mining emerged which extracts information from massive data sets

Large datasets have algorithms with high complexity

Complexity broken into two types :

a) Complexity of Training

b) Complexity of applying trained algorithm

Human Learning

Student

Textbook

Teacher

Exam

Machine Learning

Machine

Data

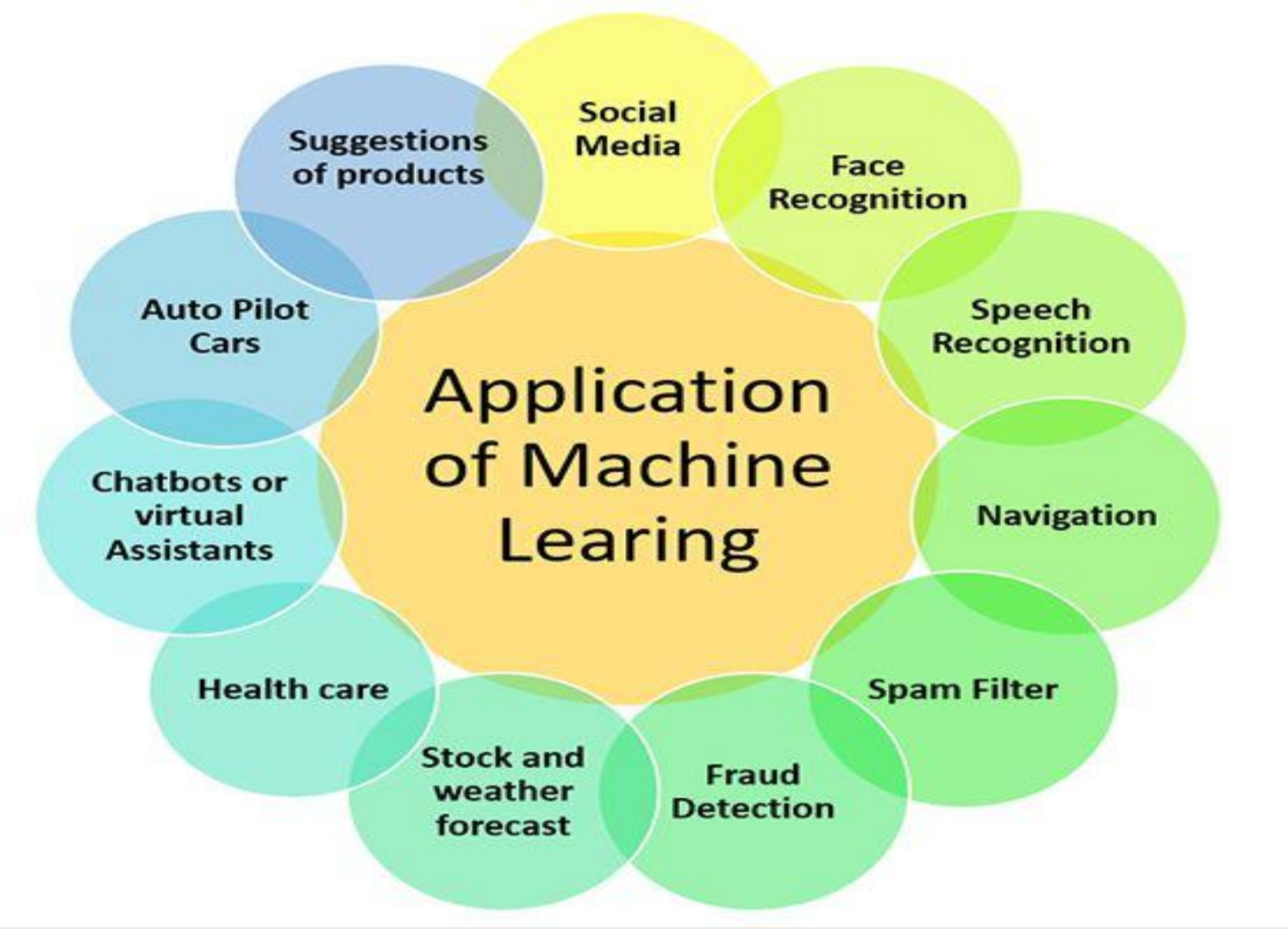
ML Algorithm

Prediction

Applications of ML

- Learning to recognize spoken words of speech recognition systems
- Learning to drive autonomous vehicles which are computer controlled
- Learning to construct astronomical structures
- Learning to play world class games like backgammon etc..

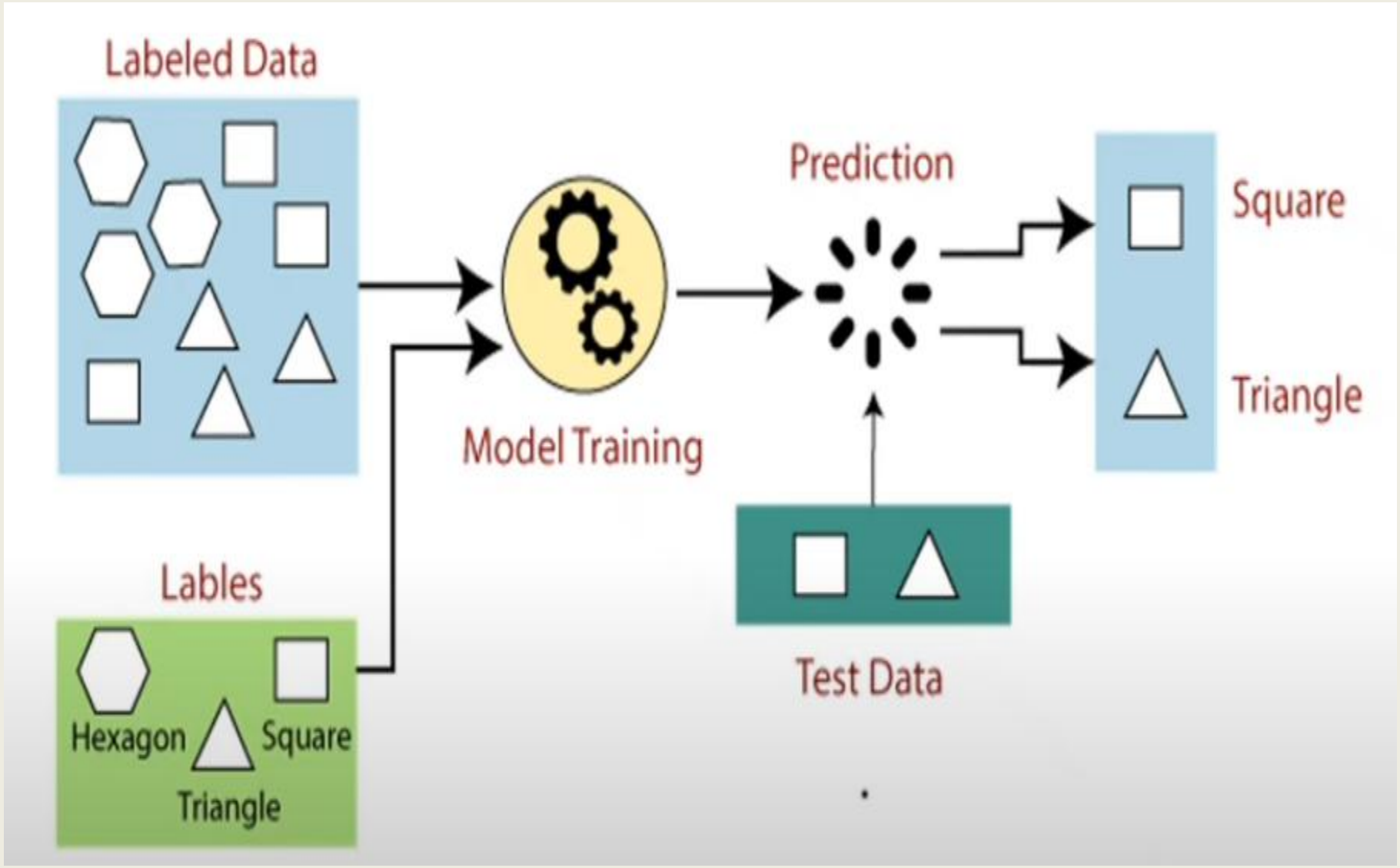
Applications of ML



Types of ML

1) Supervised learning : training with correct responses based on the training set. Algorithm responds to all inputs
2) Unsupervised learning : correct responses are not provided .i/p should have some thing in common. Statistical approaches are used called as density estimators .
3) Reinforcement learning : b/n supervised and unsupervised .Algorithm is told it is wrong .Different possibilities have to be tried .does not suggest improvement .
4) Evolutionary Learning : biological evolution is seen as learning process .It improves survival rate . It also suggests how good the solution is .

1.Supervised Learning



1.Supervised Learning

- There are two types :a) Regression b) Classification
- A set of data (training) consists of set of input –target data
- Set of data (xi, ti) , i = 1 to N

- ML is used for learning process
- **Generalization**: which solution is better to test and how we pick the values b/n data points.

a) Regression : suppose we take X ,Y,t st. X->Y, be the scalars

1.Supervised Learning

Regression is a statistical approach to find the correlations between variables (dependent and independent)

- Regression algorithms gives you a continuous output. That means if you are asking to build model that predict the future outcome where the output will be continuous. Then you must choose one of the regression algorithms to build model
- Ex: Weather forecasting, Market Trends, etc.
- Below are some popular Regression algorithms which come under supervised learning:
    - Linear Regression
    - Regression Trees
    - Non-Linear Regression
    - Bayesian Linear Regression
    - Polynomial Regression

1.Supervised Learning

Applications of Regression:

Predicting prices: For example, a regression model could be used to predict the price of a house based on its size, location, and other features.
Forecasting trends: For example, a regression model could be used to forecast the sales of a product based on historical sales data and economic indicators.
Identifying risk factors: For example, a regression model could be used to identify risk factors for heart disease based on patient data.
Making decisions: For example, a regression model could be used to recommend which investment to buy based on market data.

b)classification

- A decision points separated by decision boundaries .
- **Curse of dimensionality** : as no of inputs increases ,training of classifier takes longer as the no. of i/p dim grows .

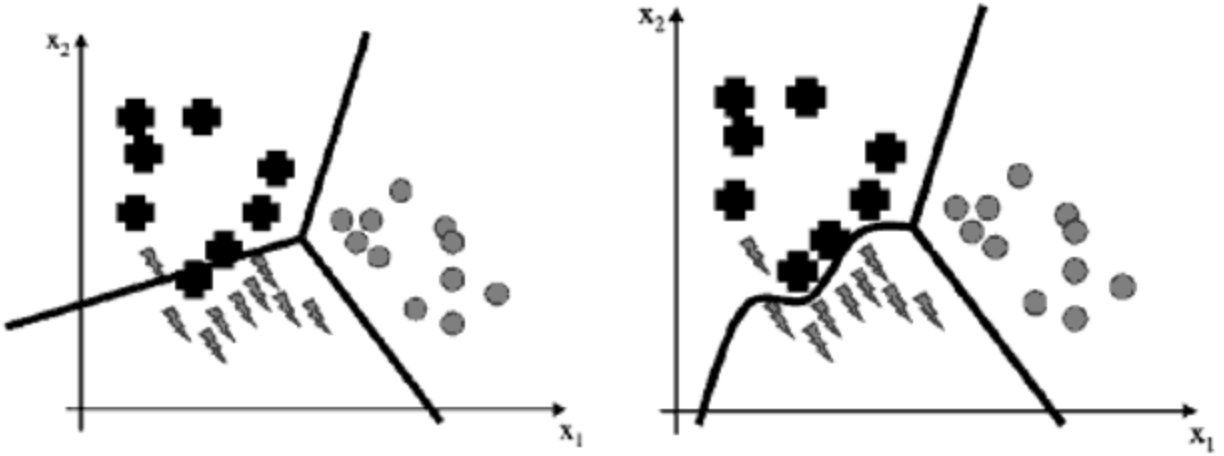


FIGURE 1.5: *Left:* A set of straight line decision boundaries for a classification problem. *Right:* An alternative set of decision boundaries that separate the plusses from the lightening strikes better, but requires a line that isn't straight.

- Classification algorithms are used when the output variable is categorical, which means there are two classes such as Yes-No, Male-Female, True-false, etc.
- Classification algorithms:
    - Random Forest
    - Decision Trees
    - Logistic Regression
    - Support vector Machines

2.Unsupervised Learning

❑Unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things

❑The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format**.

❑An unlabeled input data, which means it is not categorized and corresponding outputs are also not given.

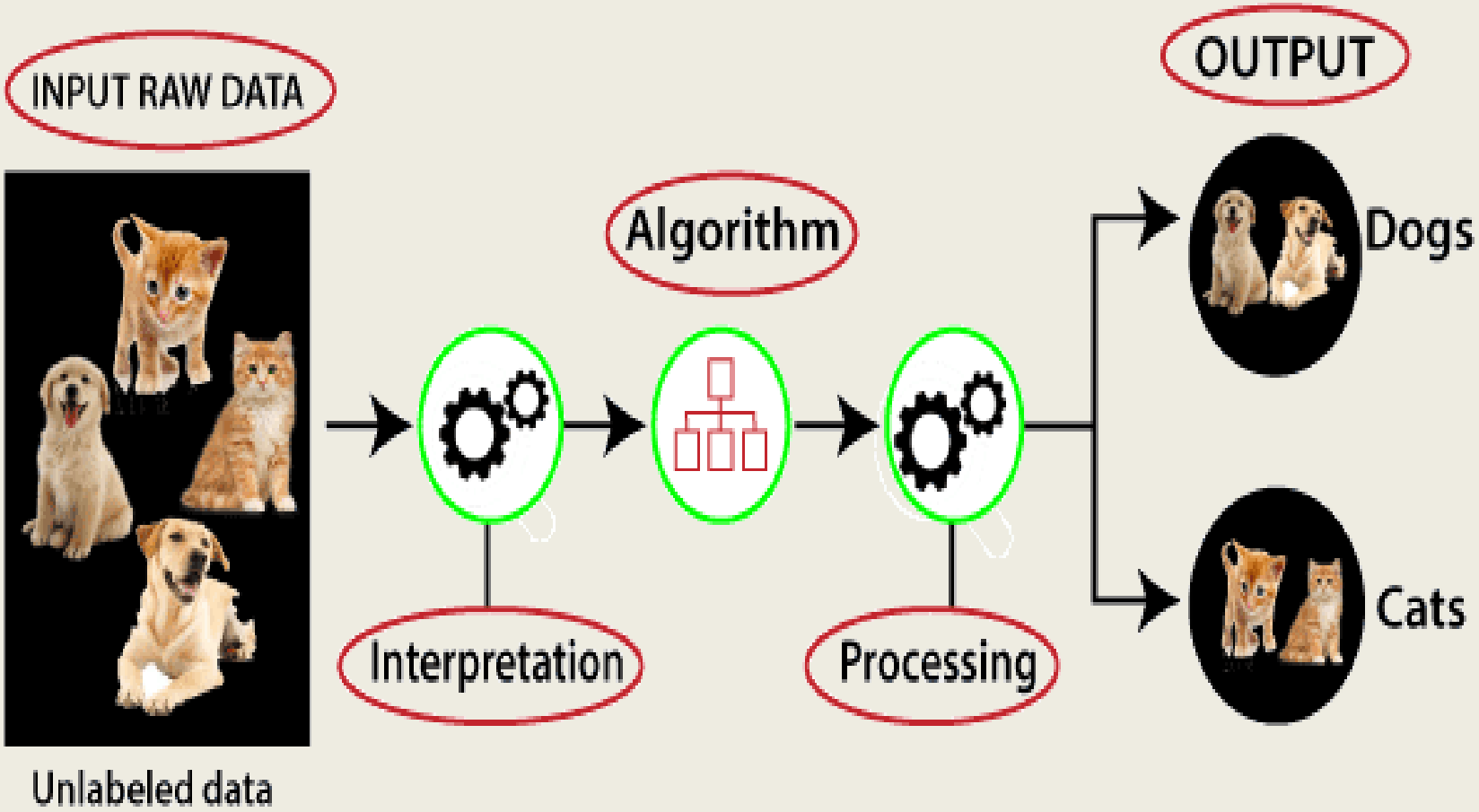
❑ This unlabeled input data is fed to the machine learning model in order to train it.

2.Unsupervised Learning

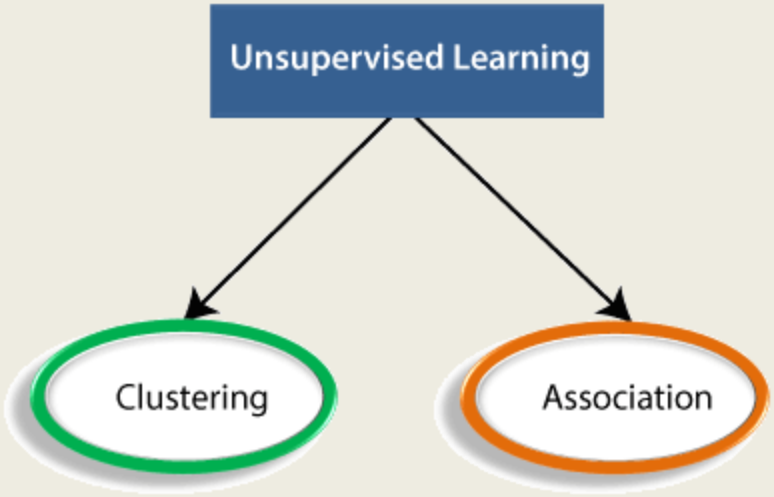
- Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

- Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

2.Unsupervised Learning



- Types of Unsupervised Learning Algorithm:



Clustering: Clustering is a method of grouping the objects into clusters such that objects with most similarities remains into a group and has less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.

- **Association**: An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Unsupervised Learning algorithms:

- **K-means clustering**
- **KNN (k-nearest neighbors)**
- **Hierarchal clustering**
- **Anomaly detection**
- **Neural Networks**
- **Principle Component Analysis**
- **Independent Component Analysis**

3.Reinforcement Learning

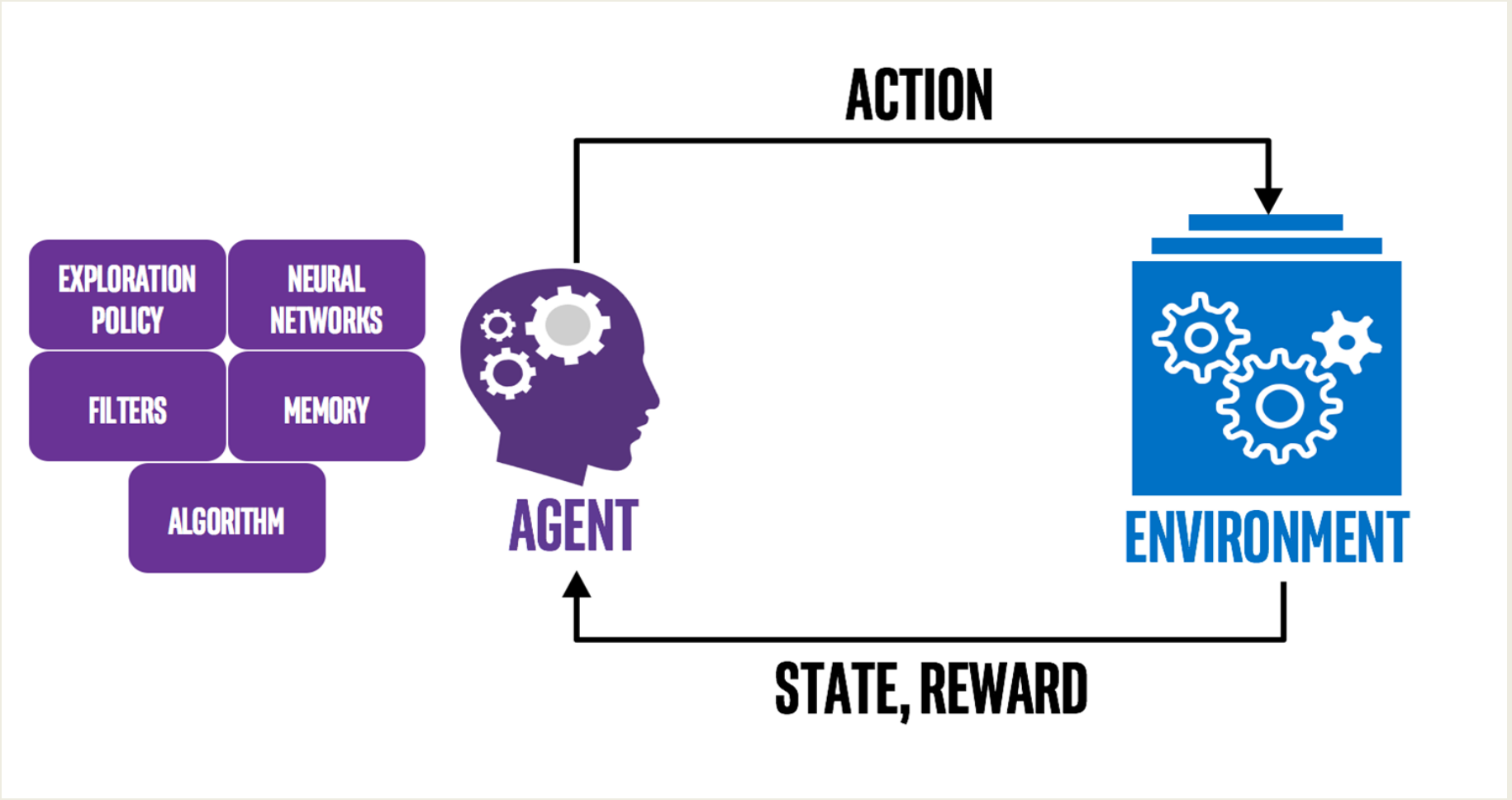
- Reinforcement Learning is a feedback-based Machine learning technique in which an agent learns to behave in an environment by performing the actions and seeing the results of actions. For each good action, the agent gets positive feedback, and for each bad action, the agent gets negative feedback or penalty.

- In Reinforcement Learning, the agent learns automatically using feedbacks without any labeled data, unlike supervised learning

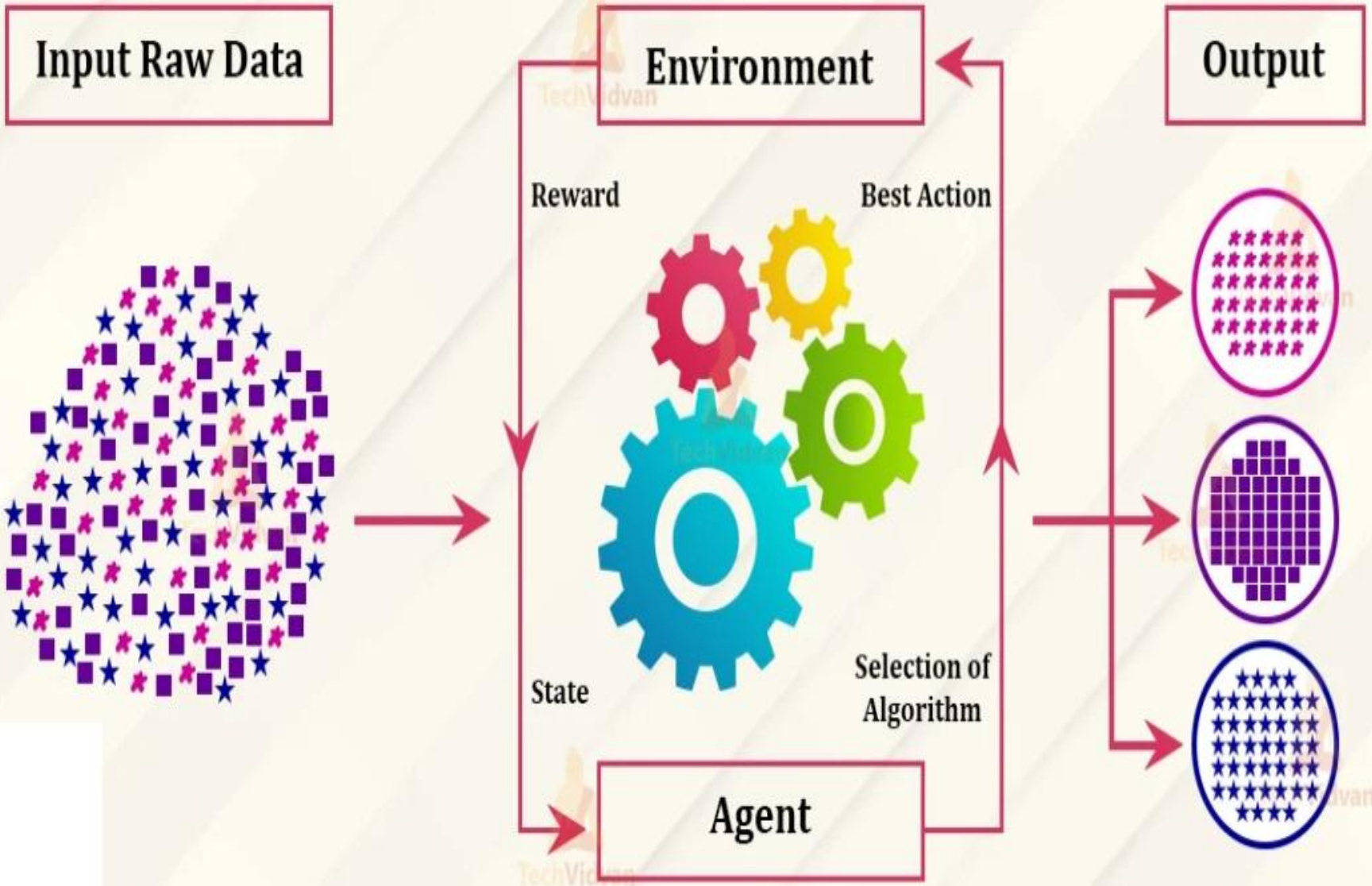
- Since there is no labeled data, so the agent is bound to learn by its experience only

3.Reinforcement Learning

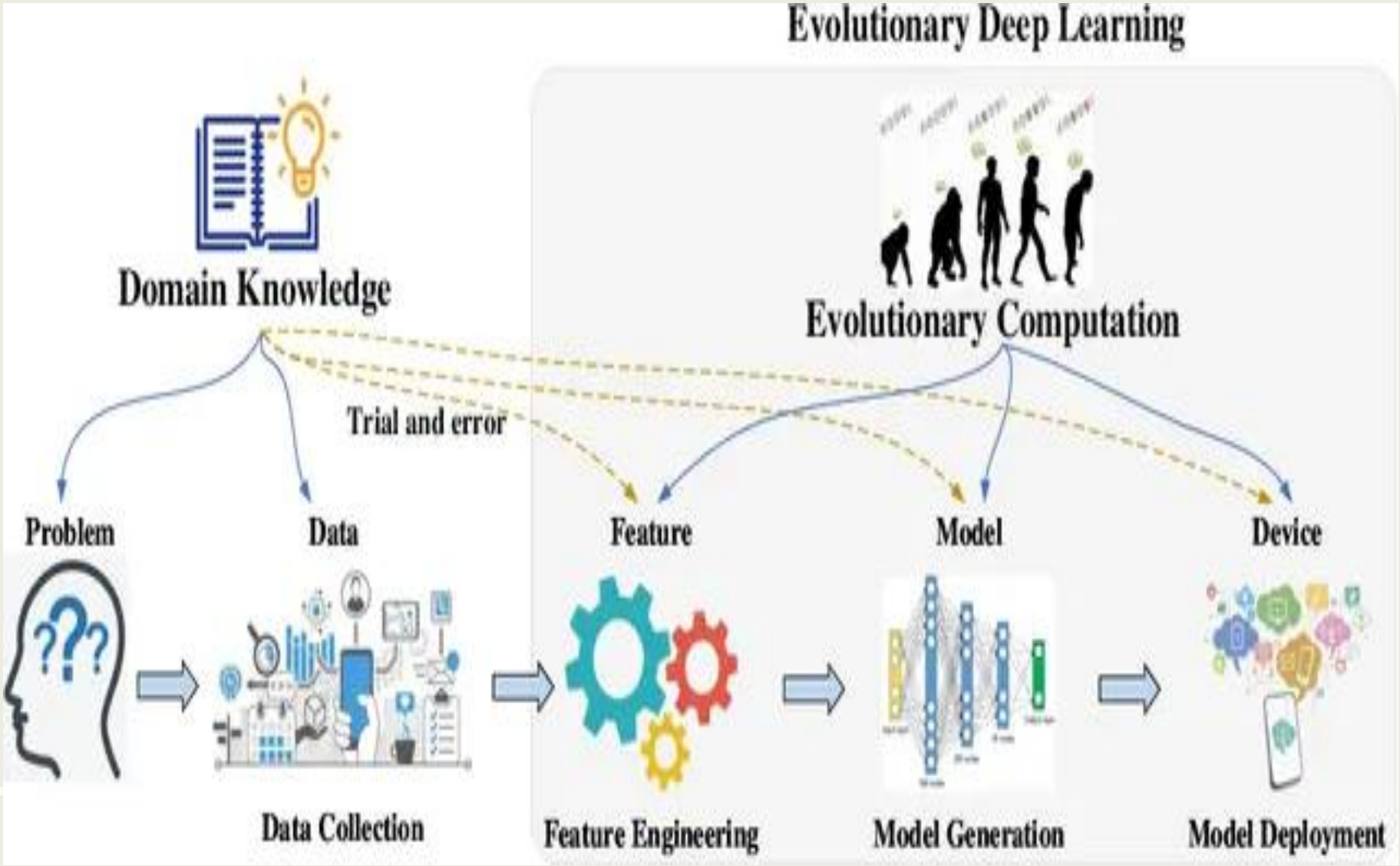
- Ex: Automatic car driving , robotic dog etc..



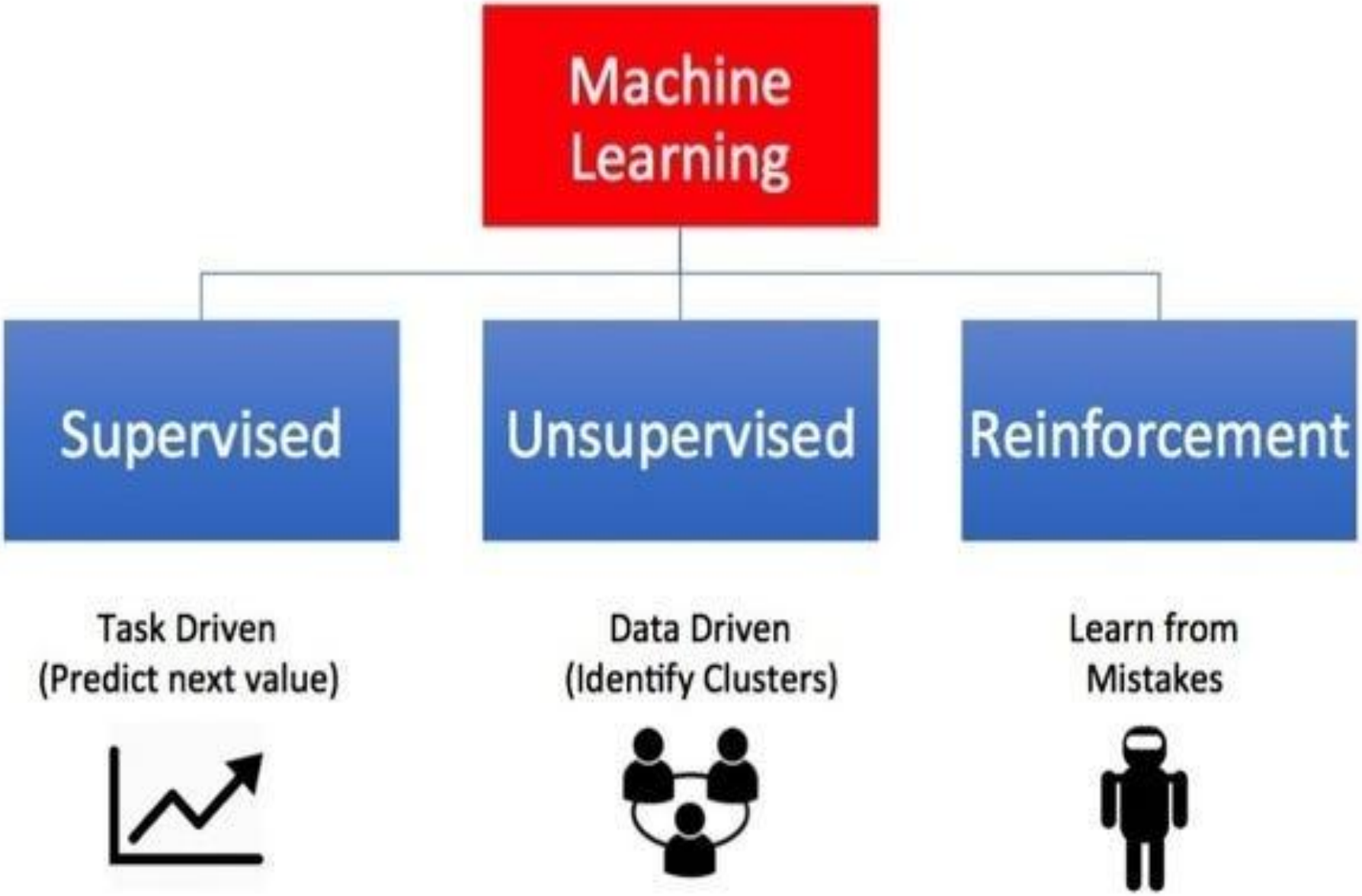
Reinforcement Learning in ML



4. Evolutionary Learning



Types of Machine Learning



BRAIN AND THE NEURON

The **brain** deals with

1. What we want noise
2. Inconsistent data
3. Produces answers

Neurons : die as we grow in age without degrade in performance

- Processing units of the brain are neurons (nerve cells) (100 billions = 10^{11})
- Transmitters : contain chemicals within the fluid of the brain (↑) or (↓) in electrical potential within neuron.
- If this membrane potential reaches some threshold –neuron spikes or fires.
- A pulse of fixed duration is sent down the axon.

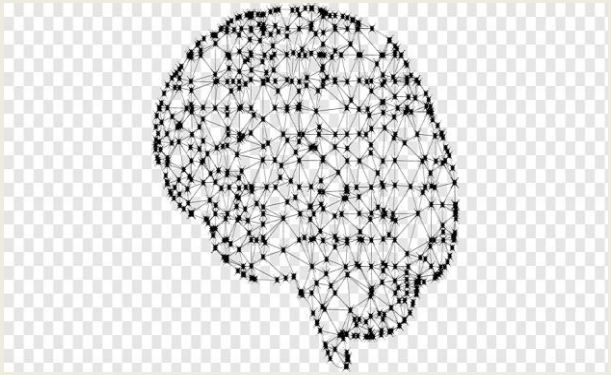
Axon : divides (arborise) into connections and connecting to each of these neurons in a synapse.

- Each neuron is connected to 100 trillion (=10^{14}) synapses
- Neuron must wait for some time to recover its energy (refractory period) before it can fire again .

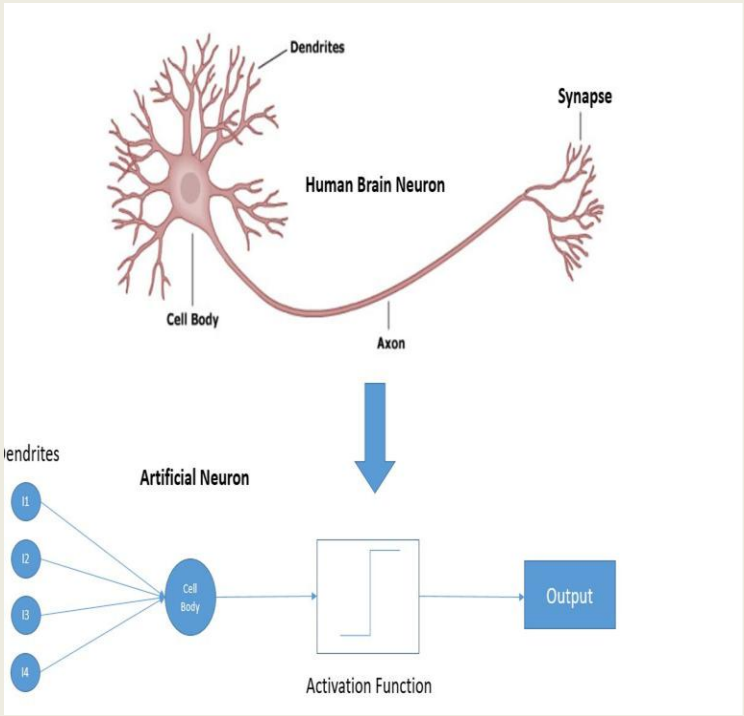
Learning : occurs in the brain using the principal concept as plasticity.

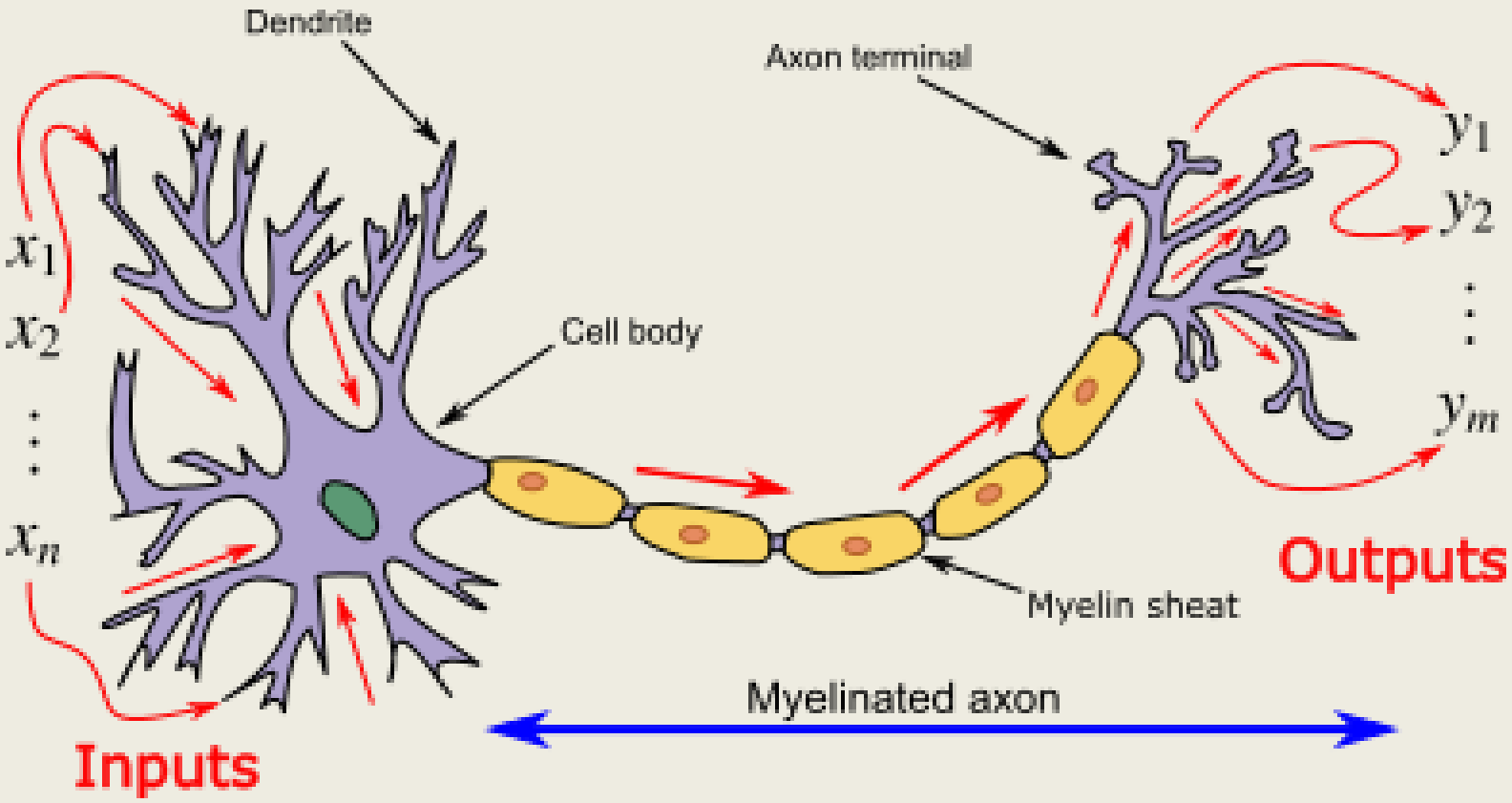
Plasticity : modify the strength of the synaptic connection between neurons thus creating new connection.

see a flower from eyes

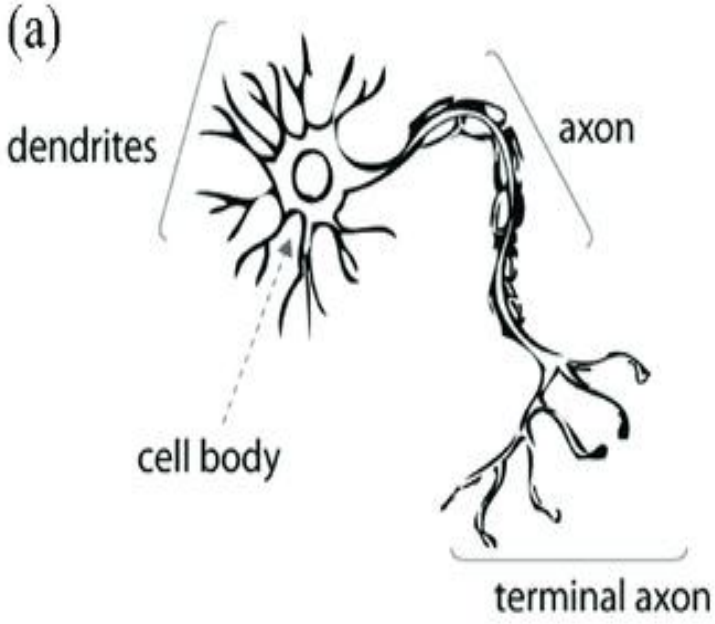


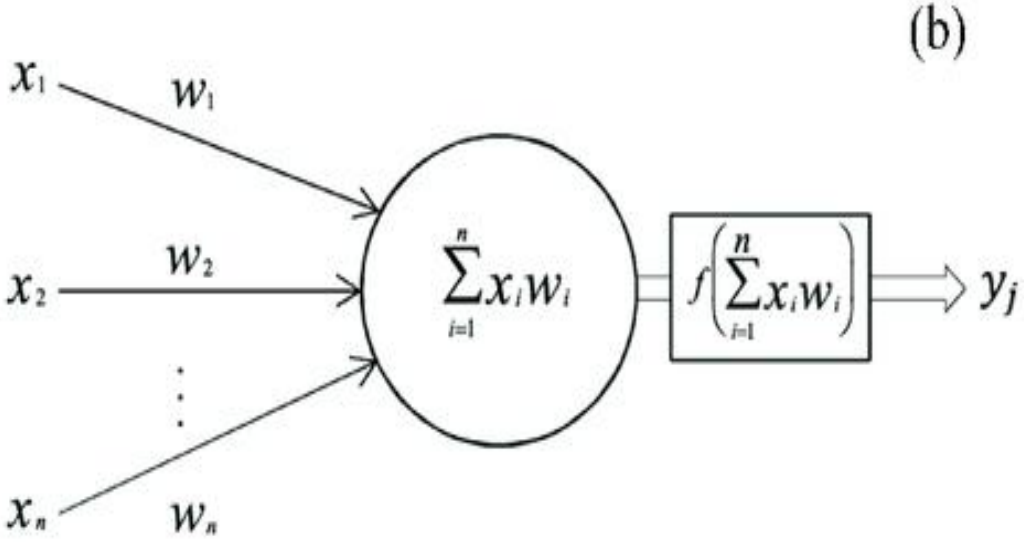
Brain

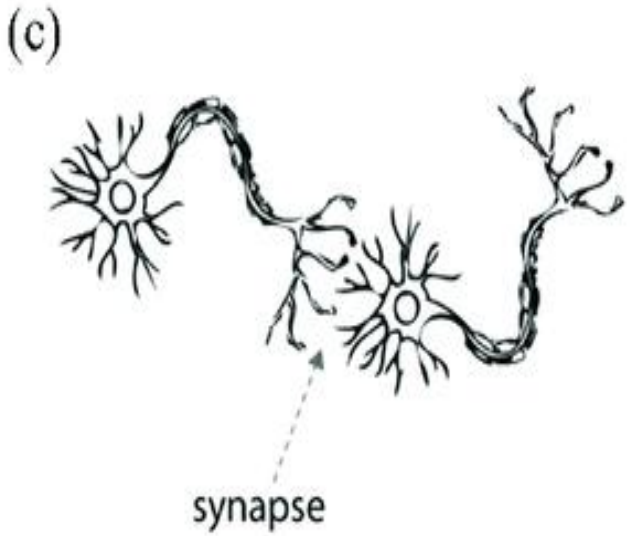


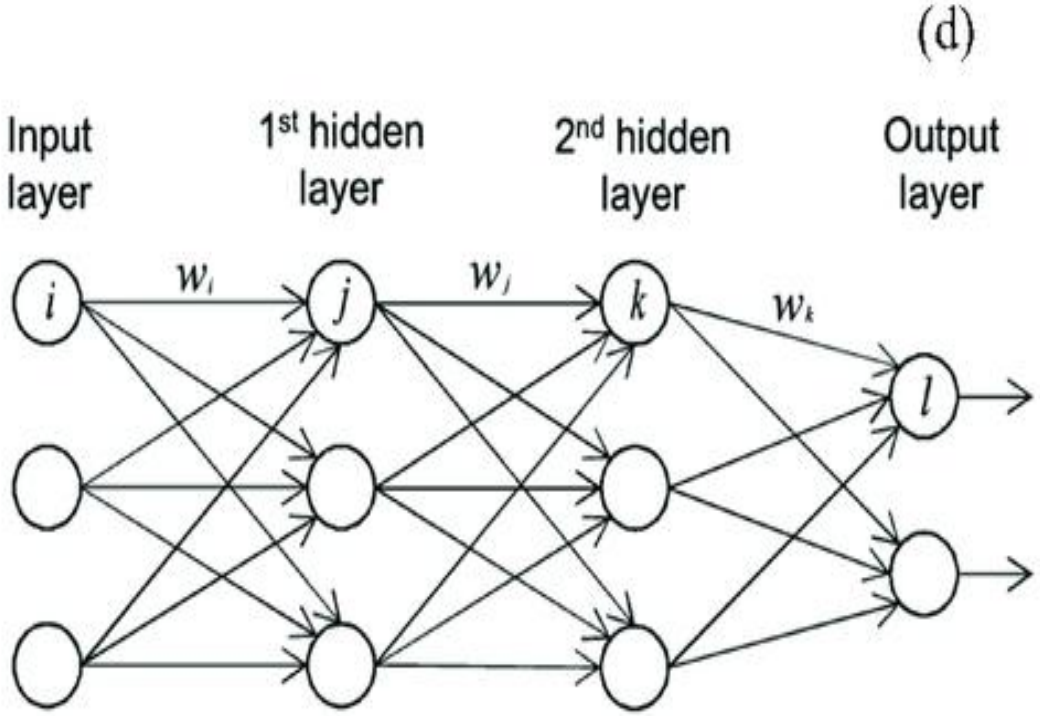


Neuron and myelinated axon, with signal flow from inputs at dendrites to outputs at axon terminals


(a)
dendrites
axon
cell body
terminal axon


(b)
x_1
w_1
x_2
w_2
x_n
w_n
$\sum_{i=1}^{n} x_i w_i$
$f\left(\sum_{i=1}^{n} x_i w_i\right)$
y_j


(c)
synapse


(d)
Input layer
1st hidden layer
2nd hidden layer
Output layer
i
j
k
l
w_i
w_j
w_k

Methods

a)Hebb's Rule :

- Introduced by Donald Hebb
- Change in the strength of the synaptic connections are 'α'(correlation in firing of 2 connecting neurons)else the neurons will die. It is a learning rule that describes how the neuronal activities influence the connection between neurons

Hebbian Learning Rule Algorithm :

1. Set all weights to zero, wi = 0 for i=1 to n, and bias to zero.

2. For each input vector, S(input vector) : t(target output pair), repeat steps 3-5.

3. Set activations for input units with the input vector Xi = Si for i = 1 to n.

4. Set the corresponding output value to the output neuron, i.e. y = t.

5. Update weight and bias by applying Hebb rule for all i = 1 to n:

Implementing AND Gate :

$$w_i(new) = w_i(old) + x_i y$$

$$b(new) = b(old) + y$$

	INPUT			TARGET	
	x_1	x_2	b		y
X_1	-1	-1	1	Y_1	-1
X_2	-1	1	1	Y_2	-1
X_3	1	-1	1	Y_3	-1
X_4	1	1	1	Y_4	1

Step 1 :

Set weight and bias to zero, $w = [\ 0\ 0\ 0\ ]^T$ and $b = 0$.

Step 2 :

Set input vector $X_i = S_i$ for $i = 1$ to 4.

$X_1 = [\ -1\ -1\ 1\ ]^T$

$X_2 = [\ -1\ 1\ 1\ ]^T$

$X_3 = [\ 1\ -1\ 1\ ]^T$

$X_4 = [\ 1\ 1\ 1\ ]^T$

Step 3 :

Output value is set to $y = t$.

Step 4 :

Modifying weights using Hebbian Rule:

First iteration –

$w(new) = w(old) + x_1y_1 = [\ 0\ 0\ 0\ ]^T + [\ -1\ -1\ 1\ ]^T\ .\ [\ -1\ ] = [\ 1\ 1\ -1\ ]^T$

For the second iteration, the final weight of the first one will be used and so on.

Second iteration –

$w(new) = [\ 1\ 1\ -1\ ]^T + [\ -1\ 1\ 1\ ]^T\ .\ [\ -1\ ] = [\ 2\ 0\ -2\ ]^T$

Third iteration –

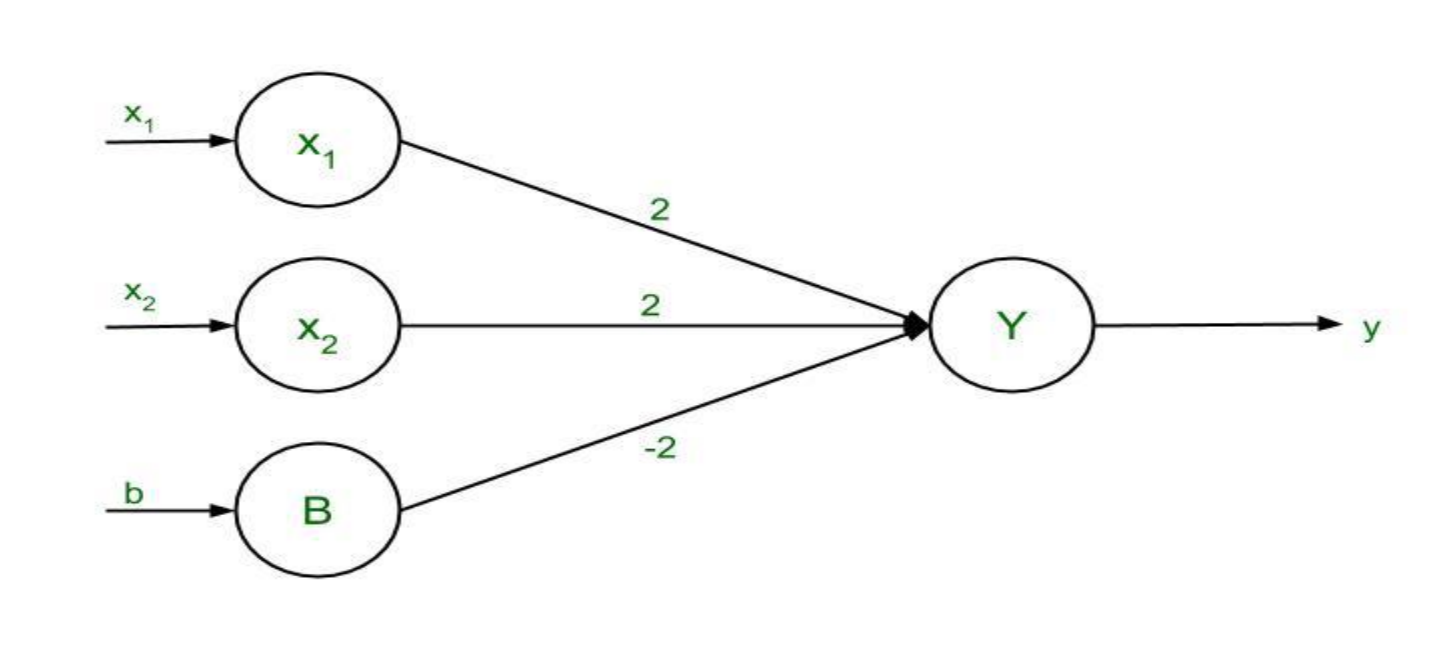
$w(new) = [\ 2\ 0\ -2]^T + [\ 1\ -1\ 1\ ]^T\ .\ [\ -1\ ] = [\ 1\ 1\ -3\ ]^T$

Fourth iteration –

$w(new) = [\ 1\ 1\ -3]^T + [\ 1\ 1\ 1\ ]^T\ .\ [\ 1\ ] = [\ 2\ 2\ -2\ ]^T$

So, the final weight matrix is $[\ 2\ 2\ -2\ ]^T$

Testing the Network:



For x1 = -1, x2 = -1, b = 1, Y = (-1)(2) + (-1)(2) + (1)(-2) = -6

For x1 = -1, x2 = 1, b = 1, Y = (-1)(2) + (1)(2) + (1)(-2) = -2

For x1 = 1, x2 = -1, b = 1, Y = (1)(2) + (-1)(2) + (1)(-2) = -2

For x1 = 1, x2 = 1, b = 1, Y = (1)(2) + (1)(2) + (1)(-2) = 2

The results are all compatible with the original table.

b)Pavlov : “grandmother gives a chocolate to her dogs” called classical conditioning

- Dogs are trained with food by ringing a bell
- The strength of synapse b/n hearing bell and saliva reflex is fired.

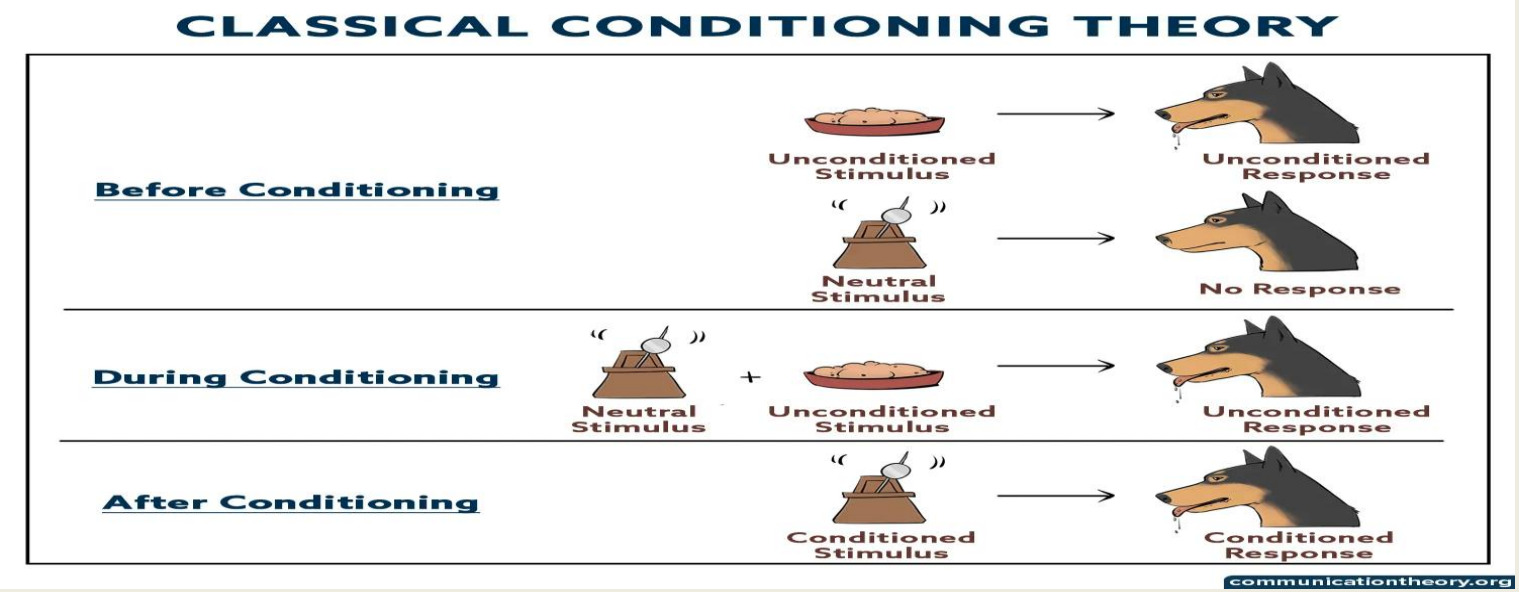
Stimulus- something that exists in the surroundings or that is presented

Response- How the animal responds to the stimulus

1. The unconditioned stimulus (US)
2. The Unconditioned Response (UR)
3. The neutral stimulus (NS)
4. The Conditioned Stimulus (CS)
5. The Conditioned Response (CR)

Ex:

When a dog presented with a meat naturally salivates to the meat, naturally salivates to the meat especially when hungry



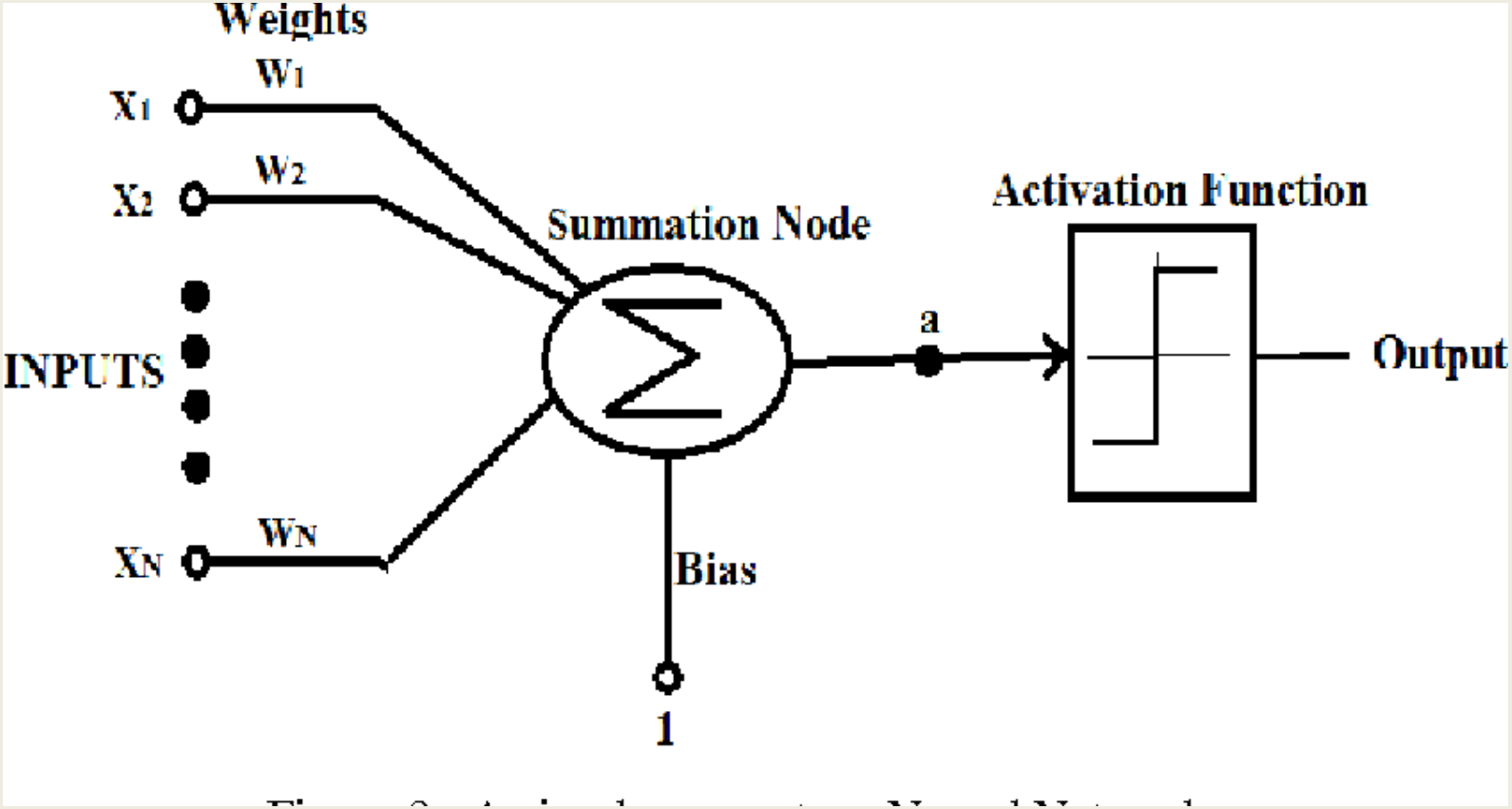
c) Mc Culloch and Pits Neurons : (1943)

- Mc culloch -pitts neuron was the earliest neuran n/w discovered in 1943
- Also called M-P neuron
- A set of weighted inputs (wi) coresp to synapse
- An adder sums the i/p signals (=membrane of cell collecting electrical changes)
- It has two layers i) input layer ii) output layer
- since the firing of the o/p neuron is based on the threshold, the activation function here is defined as

$$f(yin) = \begin{cases} 1 \text{ if } yin >= \theta \\ 0 \text{ if } yin < \theta \end{cases}$$

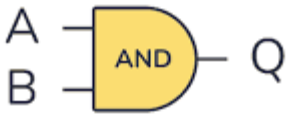
Activation Function (threshold function)

- To decide whether the neurons fire (spikes) for the inputs given



- Let the inputs be (x1,x2 …. ,xm) where
  if x_1 = 1(fired)
  x_2 = 0(didn't fire)
  x_3 = 0.5(no biological significance)

- Strength of the synapse = ($x_1w_1+x_2w_2+$ … ,x_m …
- The formula is $h = \Sigma (w_i x_i)$



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

The threshold value should satisfy the following condition:

$$\theta > nw-P$$

n → no. of neurons in input
w → positive weight
p → negative weight

Hence , Mc culloch and pitts is a binary threshold device .

Yin - $x_1w_1+x_2w_2+ \dots ,x_mw_m$

(1,1) = 1*1 + 1*1 =2
(1,0) = 1*1 + 0*1 =1
(0,1) = 0*1 + 1*1 =1
(0,0) = 0*1 + 0*1 =0

Now need to find out the threshold value in such a way the y neuron should fire as per the threshold value.

Threshold value is set equal to 2 (θ =2)

θ >= nw -p

n → no. of neurons in input (2)
w → positive weight (1)
p → negative weight (0)

n= 2, w =1 and p=0

Substituting the above values in the mentioned equation, we get

$\theta >= 2*1-0 = 2$

$\theta >= 2$

thus, the output of neuron Y can be written as

$$Y = f(yin) = \begin{cases} 1 \text{ if } yin >= 2 \\ 0 \text{ if } yin < 2 \end{cases}$$

Limitations of Mac Culloch and Pitts Neural Model

- Neurons are more complicated
- Inputs need not be linear
- Neuron gives a spike train of output (2 outputs)
- They give graded output in continuous way
- Firing the changes over time
- They are biochemical devices hence the amt of charge give out can vary.
- Neurons are updated randomly(asynchronous).
- Weights are (+ve (excitatory) & –ve (inhibitory))
- Weights change from +ve to –ve
- Links (synapse) are in a feedback loop but not possible in the network of loops.

Designing A Learning System

1. **Choosing the training system**
   1.1) whether the training experience provides –direct / indirect feedback regarding the choices made by performance of the system
   1.2) Degree to which the learner controls the sequence of training examples
   1.3) How well it represents distribution of examples
2. **Choosing the target function**
3. **Choosing a representation for the target function**
4. **Choosing a Function Approximation Algorithm**
   4.1) Estimating training values
   4.2) Adjusting the weights
5. **The final design**

Designing A Learning System

1) Choosing the training system:

Type of training experience available :It has impact on success or failure of the learner .

Key attributes :

i) whether the training experience provides –direct / indirect feedback regarding the choices made by performance of the system.

Eg: Checkers game : system learns from direct training (board states ,correct move sequence , final outcome)

Credit assignment : degree to which each move in the sequence deserves credit/blame for final outcome .

Learning from direct training is easier than indirect training .

ii) Degree to which the learner controls the sequence of training examples :

- Relies on the teacher (board states and correct moves)
- Learner has control over (board states and indirect training classification)

iii) How well it represents distribution of examples

P-performance must be measured (%)of win

If E: training experience consists of (games played against itself)

- Distribution of training examples = testing examples

Learning Problem

Task T : playing checkers

P : % of games won in the tournament

E : games played against itself

Choice :

1. Exact type of knowledge to be learned
2. Representation for target knowledge

2:Choosing Target Function

- Determine exactly what type of knowledge will be learned
- How it will be used by performance program?

Answer: choose best legal moves in large search space

Function ChooseMove

Notation : ChooseMove : **B->M**

B : legal board states

M : moves

Improve the ‘P’ of Task ‘T’

Evaluation Function

- Assigns a numerical score to any given board state
- Target Function = V
- Notation **V: B->R** ,

where B = legal board state

R = set of real nos.

Target value V(b) for an arbitrary state b in B

1) If b is final board state = won ,V(b) = 100
2) If b is a final board state =lost ,V(b) = -100
3) If b= drawn state , v(b) = 0 , V(b) = drawn
4) If b!=final state ,V(b) = V(b')

One must know the need to search ahead all the way to the end of the game according to (4).

The process of learning the target function is called as function approximation($\hat{V}$) .

Step:3 Choosing representation for target function

- Choose a value (V) which is very close $\hat{V}$

  $\hat{V}$ = linear combination of the following board features

x1: the number of black pieces on board
x2: no. of Red pieces
X3: black kings
X4: red kings
X5: no. of black pieces threatened by Red (captured on red in next turn)
X6: no. of red pieces threatened by black.
Learning Program: will represen $\hat{V}$ (b) as linear function of the form :-

$w_0 \ldots\ldots w_6$ = numerical coefficients /weights

$$\hat{V}(b) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + w_5 x_5 + w_6 x_6$$

4. Choosing a function Approximation Algorithm

- To learn target function $\hat{V}$ set of training examples are required .
- It specifies board state **b** as training value $V_{train}(b)$
- Each training example is an ordered pair $\langle b, V_{train}(b) \rangle$

Eg : black has won the game

($x_2 = 0$, no red is left)

- Target function value $V_{train}(b)$ $= +100$

(black has won hence +ve)

$$\langle \langle x_1 = 3, x_2 = 0, x_3 = 1, x_4 = 0, x_5 = 0, x_6 = 0 \rangle, +100 \rangle$$

4.1Estimating the training values

- In Every step, We consider the successor (Depending on the next step of opponent)

$$V_{train}(b) \leftarrow \hat{V}(Successor(b))$$

$\hat{V}$ = represents Approximation

successor(b) = next board state following ‘b’ (Estimating that this move will help/destroy opponent)

4.2 Adjusting the weights

- Choosing the weights (w_i) to best fit training examples $\langle b, V_{train}(b) \rangle$
- Solution : best define the hypothesis /weights minimizing squared error 'E' between training values and values predicted over hypothesis

$$E \equiv \sum_{\langle b, V_{train}(b) \rangle \in \text{training examples}} (V_{train}(b) - \hat{V}(b))^2$$

LMS weight update rule.

For each training example $\langle b, V_{train}(b) \rangle$

- Use the current weights to calculate $\hat{V}(b)$
- For each weight w_i, update it as

$$w_i \leftarrow w_i + \eta \, (V_{train}(b) - \hat{V}(b)) \, x_i$$

There are some algorithms to find the weights of linear functions, Here we are using Lease mean square (Used to minimize the error)

if error =0 → No need to change weights

error = positive → each weight is increased

error = negative → each weigh is decreased

5.THE FINAL DESIGN

- It has four modules

1)Performance system: solves given performance task .Next move is determined by $\hat{V}$.The performance becomes increasingly accurate.

2) Critic: Takes as the input the history or trace of the game and produces as output a set of training examples of the target function .It operates on train rule ie $V_{train}(b) \leftarrow \hat{V}(Successor(b))$

3)Generalizer : takes as input the training examples and produces an output hypothesis that is its estimation of target function.

- Experiment generator: It picks new practice problems that maximize learning rate (η) of over all system

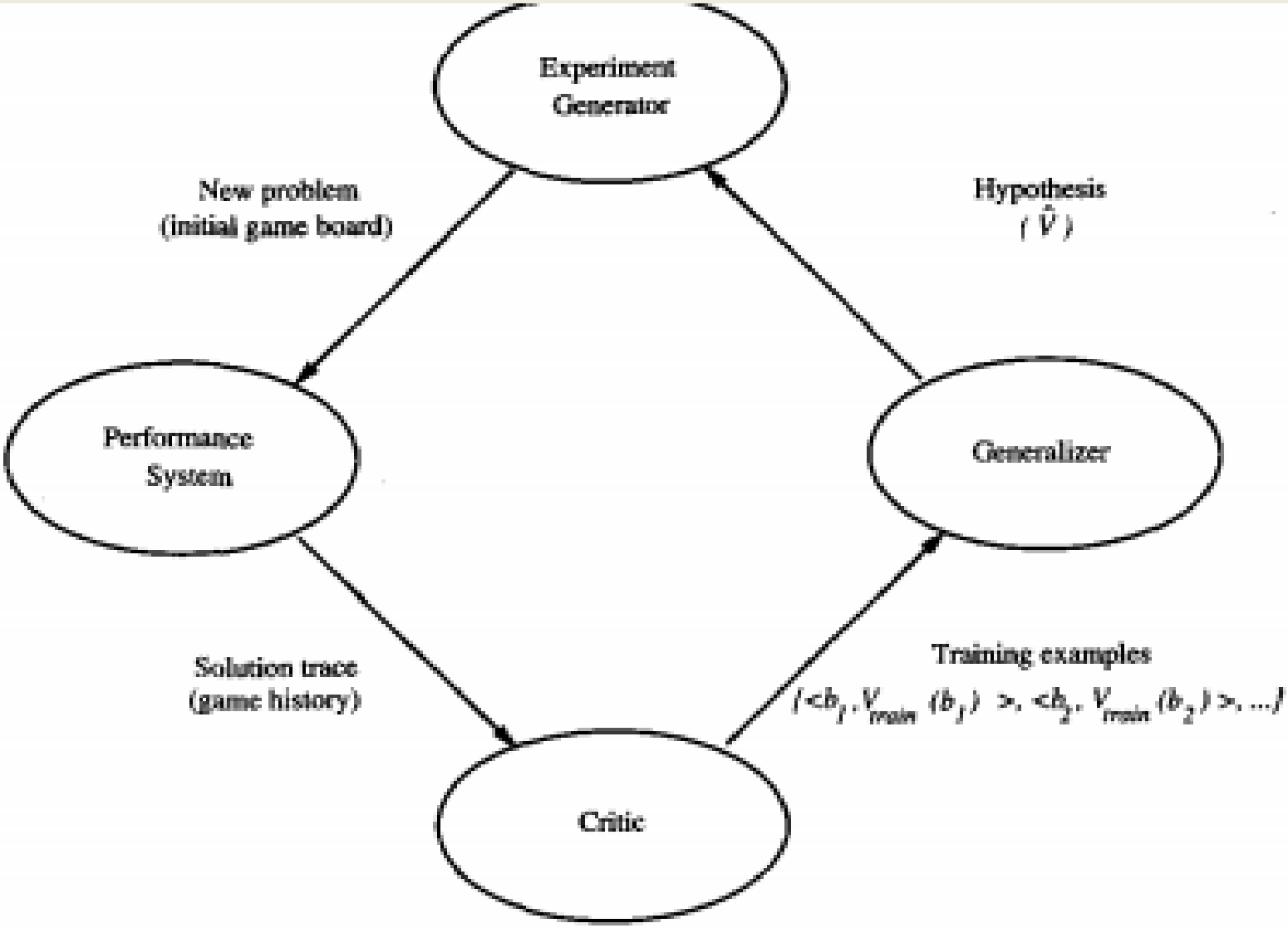


FIGURE 1.1
Final design of the checkers learning program.

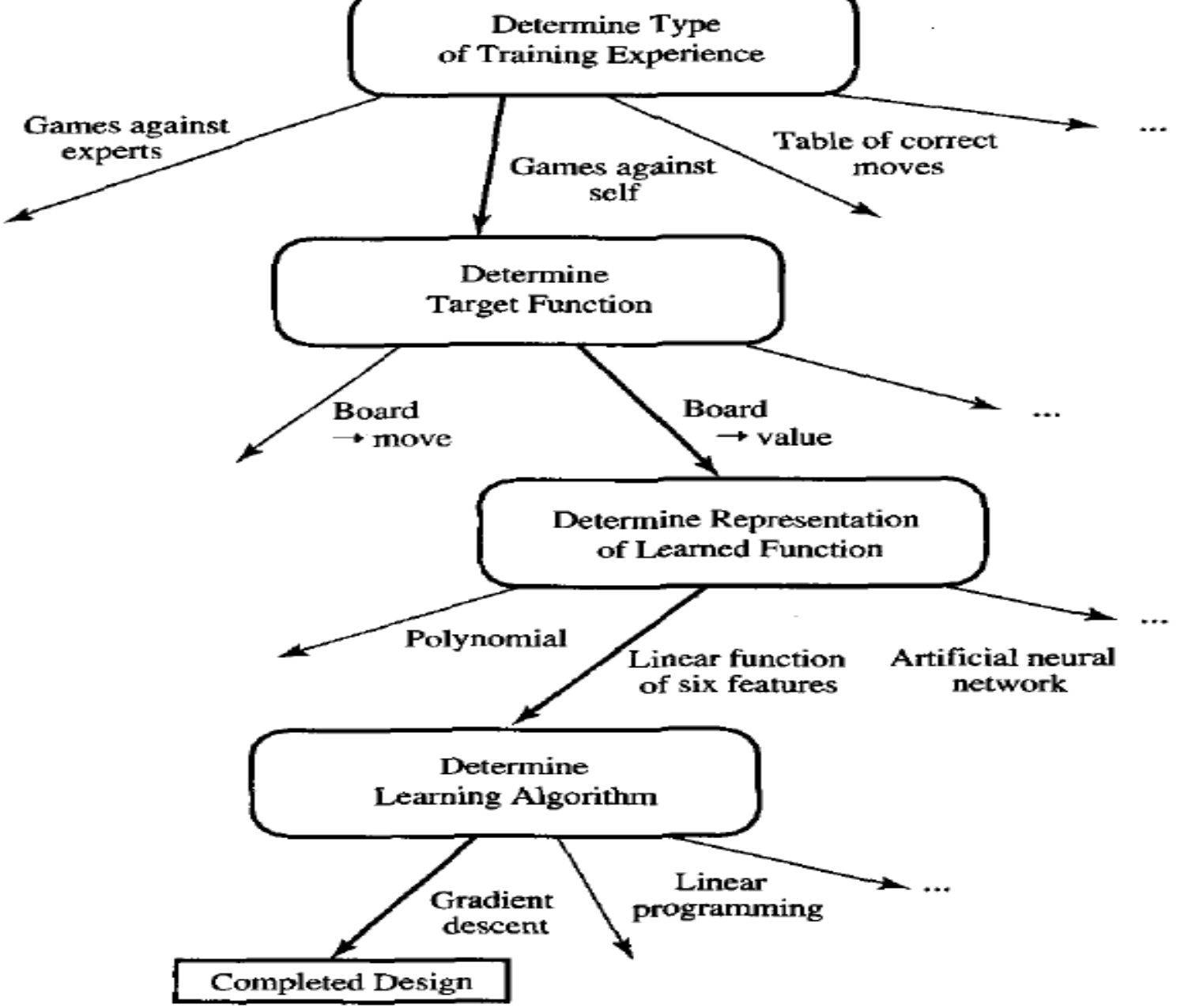


FIGURE 1.2
Summary of choices in designing the checkers learning program.

Issues in machine learning

- What algorithm exists for learning general target function from training examples.
- How much training data is sufficient?
- When and how prior knowledge held by learner guide the process
- What is the best strategy for choosing next training experience?
- How to reduce learning task ?
- How learner alter its representation in target function

Concept learning

- Defn : Every concept is a Boolean-valued function defined over larger set.
- Learning involves acquiring general concepts from specific training examples
- Each such concepts can be viewed as describing some subset of objects or events defined over a larger set.
- A task of acquiring potential hypothesis (solution) that best fits the given training examples

Concept learning

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

TABLE 2.1
Positive and negative training examples for the target concept *EnjoySport*.

Days on which my friend Aldo enjoyed his favorite water sports?

Hypothesis

- Consists of conjunction of constraints on instance attributes
- It has 6 constraints according to the table
- Indicated by '?' (any value is acceptable for this attribute)
- Indicated by single value 'warm'
- 'Φ' = no values accepted (null)
- Defn : Inference : set of items over which the concept is defined
- Let 'x' be the instance
- Let 'h' be the hypothesis
- 'x' satisfies all constraints of 'h' ie (h(x) = 1) , x is +ve

Q)Aldo enjoys his favorite sport only on cold days with high humidity.

General hypothesis : $\langle ?, ?, ?, ?, ?, ? \rangle$

Representation of expression : $\langle ?, Cold, High, ?, ?, ? \rangle$

No. of days it is positive: $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$

Instance is rep. by x = {sky,Airtemp,Humidity,Wind,water,forecast}

Target Concept(c)= concept /function to be learned .C is a Boolean valued function over x

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

TABLE 2.1
Positive and negative training examples for the target concept *EnjoySport*.

$c : X \rightarrow \{0, 1\}.$

- Target Concept corresponds to the value of the attribute. (i.e., $c(x) = 1$ if $EnjoySport = Yes$, and $c(x) = 0$ if $EnjoySport = No$).
- Ordered pair (x, c(x)) describes training example

- **Given:**
  - Instances X: Possible days, each described by the attributes
    - *Sky* (with possible values *Sunny*, *Cloudy*, and *Rainy*),
    - *AirTemp* (with values *Warm* and *Cold*),
    - *Humidity* (with values *Normal* and *High*),
    - *Wind* (with values *Strong* and *Weak*),
    - *Water* (with values *Warm* and *Cool*), and
    - *Forecast* (with values *Same* and *Change*).
  - Hypotheses H: Each hypothesis is described by a conjunction of constraints on the attributes *Sky*, *AirTemp*, *Humidity*, *Wind*, *Water*, and *Forecast*. The constraints may be "?" (any value is acceptable), "$\emptyset$" (no value is acceptable), or a specific value.
  - Target concept c: $EnjoySport : X \rightarrow \{0, 1\}$
  - Training examples D: Positive and negative examples of the target function (see Table 2.1).
- **Determine:**
  - A hypothesis h in H such that $h(x) = c(x)$ for all x in X.

Given : target concept (c)EnjoySport :X → {0,1}

D = set of available training examples

H = set of all possible hypothesis regarding the identity of target concept .

X= possible days described by the attributes

Determine : ‘h’ in ‘H’ that represents Boolean-valued functions defined over X

h = $h : X \rightarrow \{0, 1\}.$

Goal : find ‘h’ such that h(x) = c(x) for all x in X

Inductive Learning Hypothesis

- It can guarantee that the output hypothesis fits the target concept over the training data
- Hypothesis that best fits the observed training data
- Defn : any hypothesis found to approximate target function over large set of training examples .

Concept learning as a search

- Defn : searching through large space of hypothesis implicitly defined by hypothesis representation
- Goal : find the hypothesis that best fits the training examples .
- Instance space X : contains 3.2.2.2.2.2 = 96 distinct instances
- Hypothesis (H) : contains 5.4.4.4.4.4=5120 (syntactically distinct hypothesis)
- If one of the move ‘Φ’ is present in the (semantically) distinct hypothesis then 1+(4.3.3.3.3.3) = 973(if it considers every instance as **negative**)

General to specific ordering of hypothesis

- Defn : we can design learning algorithm that exhaustively searches even infinite hypothesis space without explicitly enumerating every hypothesis .
- Consider 2 hypothesis

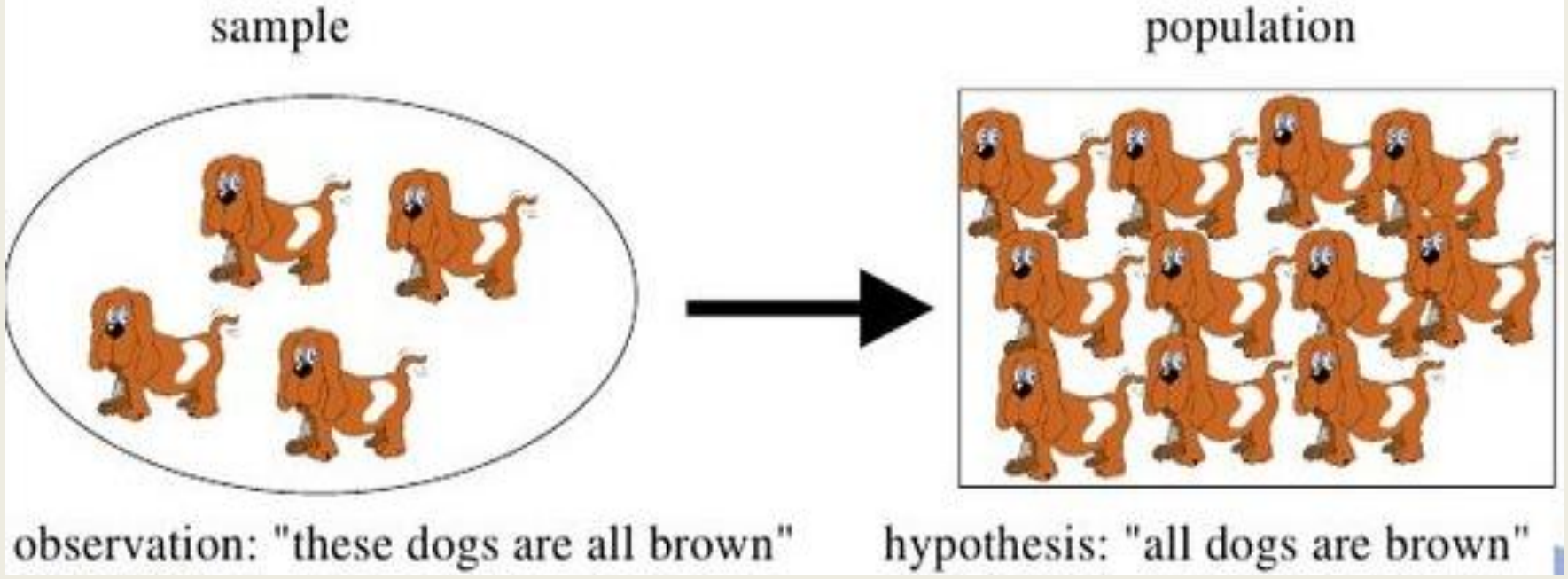
$$h_1 = \langle Sunny, ?, ?, Strong, ?, ? \rangle$$

$$h_2 = \langle Sunny, ?, ?, ?, ?, ? \rangle$$

Consider h1 and h2 are +ve(h2 imposes fewer instances and classifies more instances as +ve

Inductive inference

- = Reasoning from specific to general
- e.g. statistics: from sample, infer properties of population



For any instance : x in X
Hypothesis: h in H
X satisfies h iff: h(x) = 1

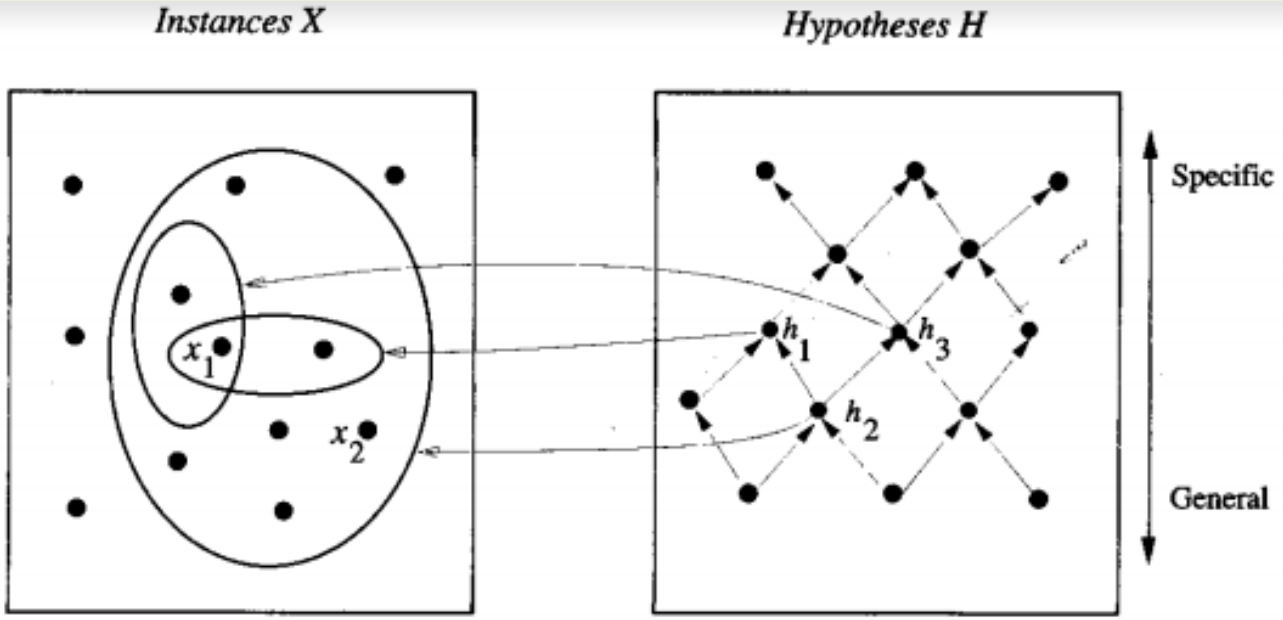
Definition: Let h_j and h_k be boolean-valued functions defined over X. Then h_j is **more_general_than_or_equal_to** h_k (written $h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$

Special case : one hypothesis is more general than others

h_j (more general _ than) h_k indicated as $h_j >_g h_k$ iff $(h_j \geq_g h_k) \wedge (h_k \not\geq_g h_j)$

$\geq_g$ Defines partial order over hypothesis space H if it is (reflexivity($x = x$.) , anti-symmetric (if (a,b)€ R and (b,a) € R then a = b) and transitivity if $x = y$ and $y = z$, then $x = z$.



x_1= <Sunny, Warm, High, Strong, Cool, Same>

x_2= <Sunny, Warm, High, Light, Warm, Same>

h_1= <Sunny, ?, ?, Strong, ?, ?>

h_2= <Sunny, ?, ?, ?, ?, ?>

h_3= <Sunny, ?, ?, ?, Cool, ?>

FIGURE 2.1

Instances, hypotheses, and the *more_general_than* relation. The box on the left represents the set X of all instances, the box on the right the set H of all hypotheses. Each hypothesis corresponds to some subset of X—the subset of instances that it classifies positive. The arrows connecting hypotheses represent the *more_general_than* relation, with the arrow pointing toward the less general hypothesis. Note the subset of instances characterized by h_2 subsumes the subset characterized by h_1, hence h_2 is *more_general_than* h_1.

Finding Maximally Specific Hypothesis [FIND-S]

- Hypothesis consists of training examples (+ve and –ve)

Most General Hypothesis: h = <?, ?, ?, ?, ?, ?>

Most Specific Hypothesis: h = < Ø, Ø, Ø, Ø, Ø, Ø>

Step 1

- Initialize h to more specific hypothesis in H
- None of the 'Φ' constraints are satisfied
- according to the example

$$h \leftarrow \langle Sunny, Warm, Normal, Strong, Warm, Same \rangle$$

- h is specific(–ve)
- Place'?' to any position (3rd).
- This is –ve(no revision is required for h

$$h \leftarrow \langle Sunny, Warm, ?, Strong, Warm, Same \rangle$$

Step 1: FIND-S

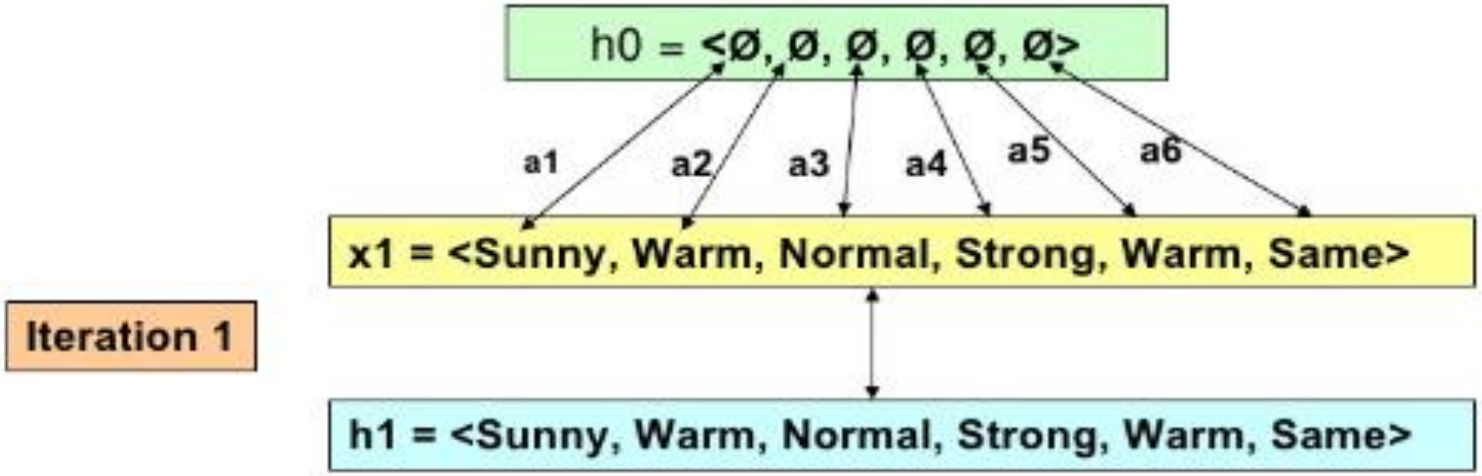
Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

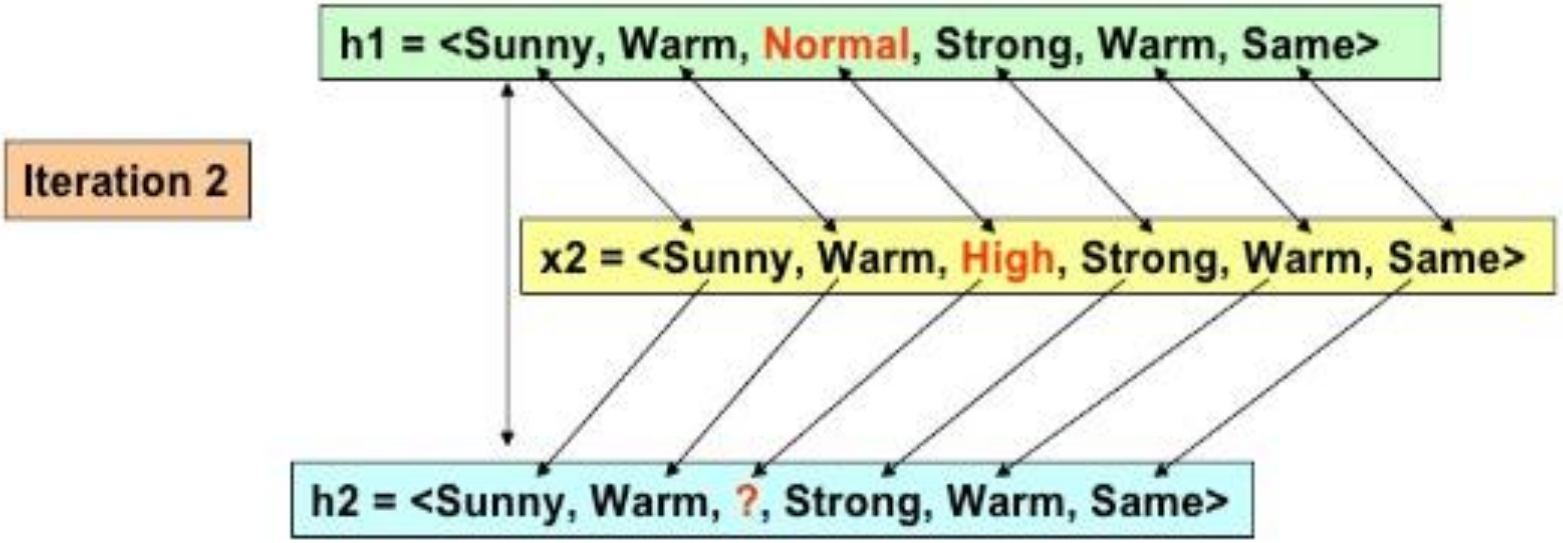
1. Initialize h to the most specific hypothesis in H

h0 = <Ø, Ø, Ø, Ø, Ø, Ø>

Step 2: FIND-S

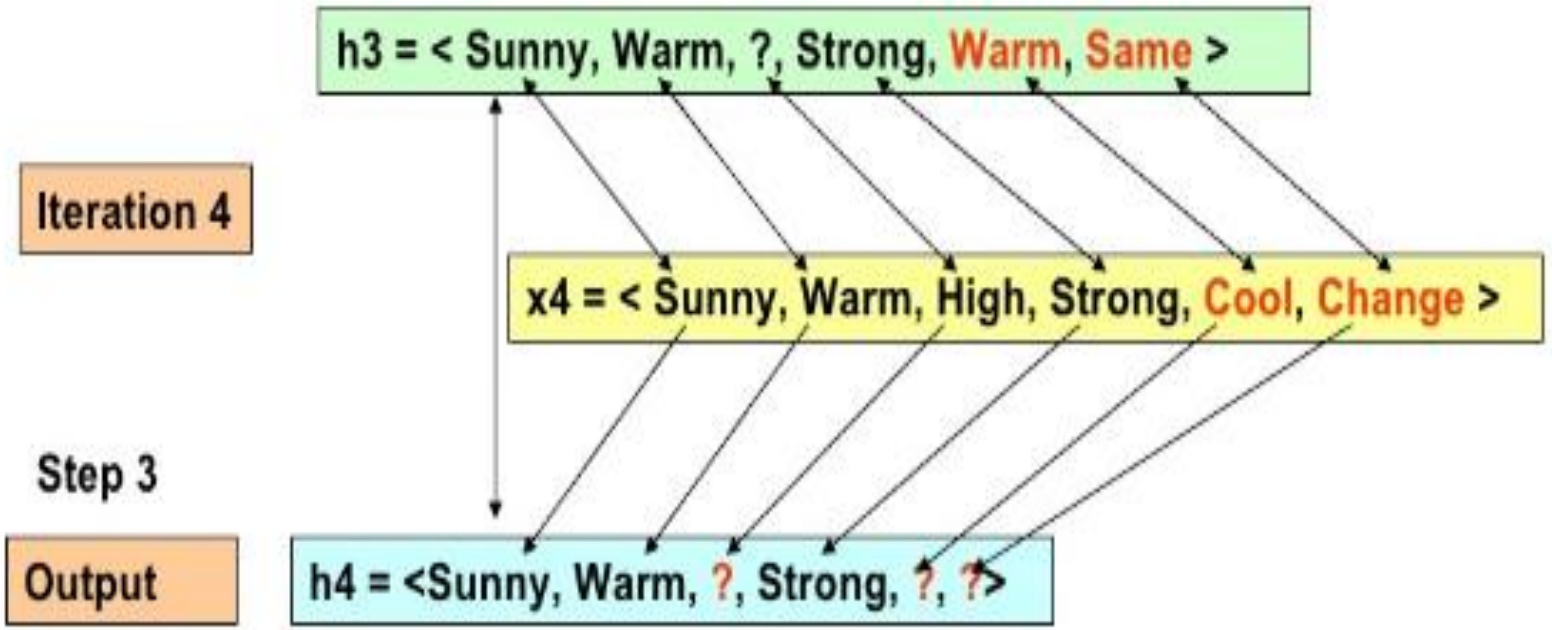
2. For each positive training instance x
   - For each attribute constraint a_i in h
     If the constraint a_i is satisfied by x
     Then do nothing
     Else replace a_i in h by the next more general constraint that is satisfied by x





Iteration 3 | Ignore | h3 = <Sunny, Warm, ?, Strong, Warm, Same>

4th training is a positive example
Search moves from h to h from most specific to most general hypothesis along partial ordering



1. Initialize h to the most specific hypothesis in H
2. For each positive training instance x
    - For each attribute constraint a_i in h

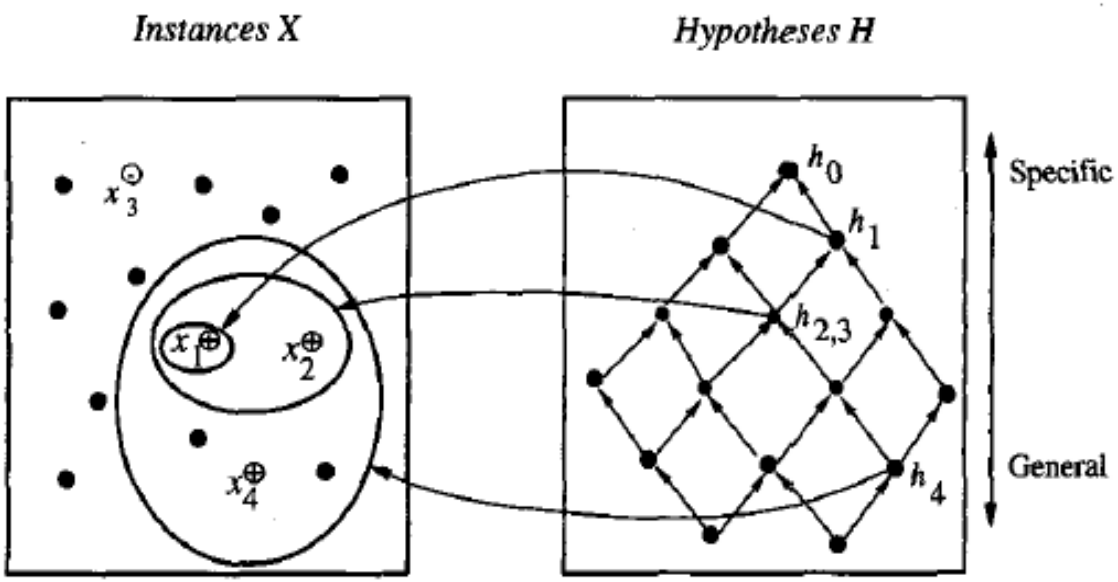
        If the constraint a_i is satisfied by x

        Then do nothing

        Else replace a_i in h by the next more general constraint that is satisfied by x
3. Output hypothesis h

TABLE 2.3

FIND-S Algorithm.



x_1 = <*Sunny Warm Normal Strong Warm Same*>, +

x_2 = <*Sunny Warm High Strong Warm Same*>, +

x_3 = <*Rainy Cold High Strong Warm Change*>, -

x_4 = <*Sunny Warm High Strong Cool Change*>, +

h_0 = <∅, ∅, ∅, ∅, ∅, ∅>

h_1 = <*Sunny Warm Normal Strong Warm Same*>

h_2 = <*Sunny Warm ? Strong Warm Same*>

h_3 = <*Sunny Warm ? Strong Warm Same*>

h_4 = <*Sunny Warm ? Strong ? ?*>

FIGURE 2.2

The hypothesis space search performed by FIND-S. The search begins (h_0) with the most specific hypothesis in H, then considers increasingly general hypotheses (h_1 through h_4) as mandated by the training examples. In the instance space diagram, positive training examples are denoted by "+," negative by "–," and instances that have not been presented as training examples are denoted by a solid circle.

Unanswered Questions by FIND-S

- Has the learner converged to the correct target concept?
- Why prefer the most specific hypothesis?
- What if the training examples consistent?

Version Space

The set of all valid hypotheses provided by an algorithm is called **version space (VS)** with respect to the hypothesis space **H** and the given example set **D**.

Definition: A hypothesis h is **consistent** with a set of training examples D if and only if $h(x) = c(x)$ for each example $\langle x, c(x)\rangle$ in D.

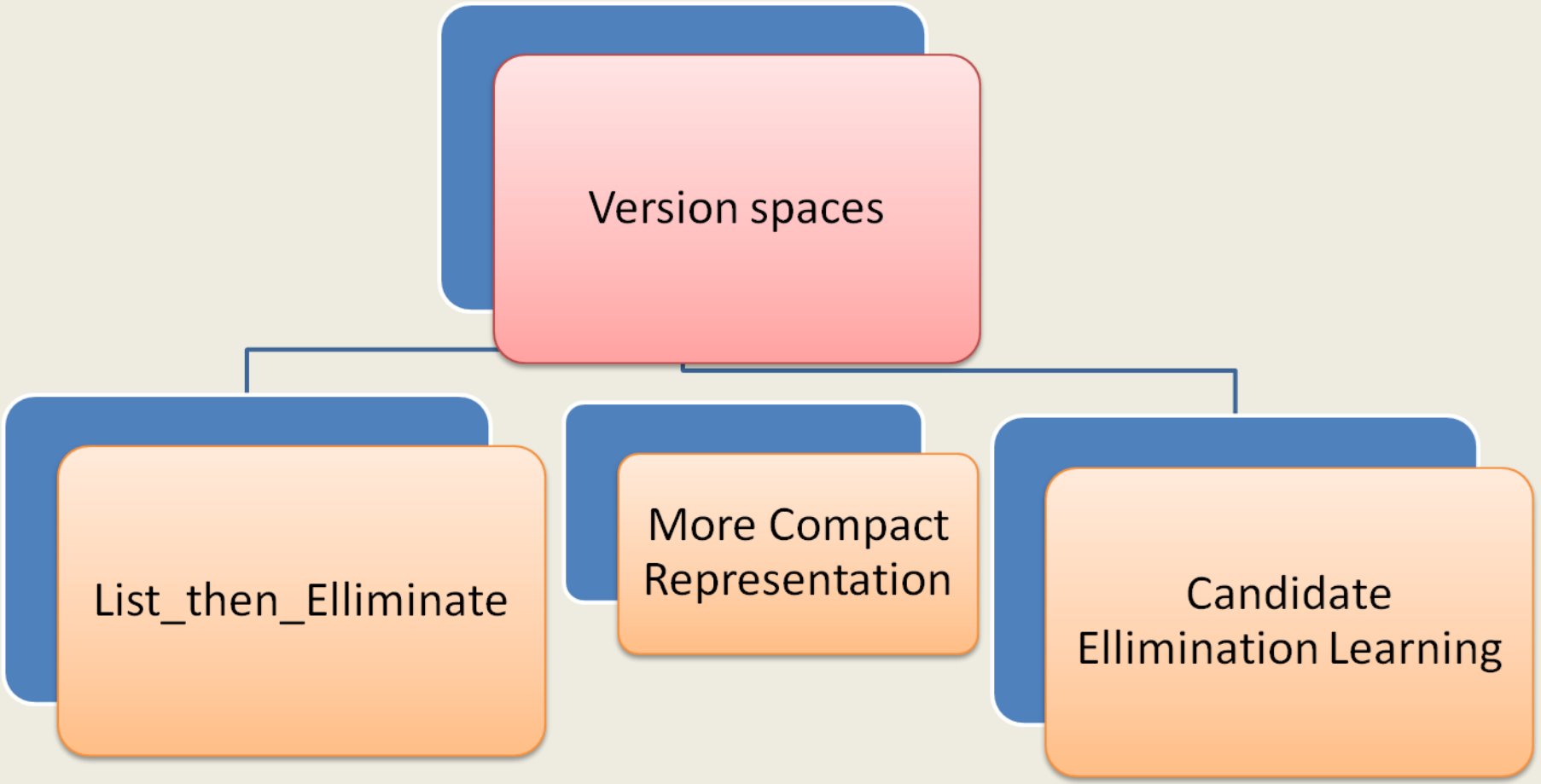
$$Consistent(h, D) \equiv (\forall \langle x, c(x)\rangle \in D)\ h(x) = c(x)$$

- No explicit enumerated members
- IT uses more _general_then partial ordering to main representation of set of H .
- The subset of all hypothesis is called as version space with respect to H and D.

Definition: The **version space**, denoted $VS_{H,D}$, with respect to hypothesis space H and training examples D, is the subset of hypotheses from H consistent with the training examples in D.

$$VS_{H,D} \equiv \{h \in H | Consistent(h, D)\}$$

Version spaces types



1.List _Then _eliminate

- List all of its members
- 1st initialize version space containing all H
- Eliminate any H focused inconsistent
- Version space shrinks as training examples are observed
- Only one H remains that is consistent with all examples giving Target Concept(c)
- If insufficient data = FOUND then output entire set of H consistent with observed data
- It can be applied when H is finite
- It output all H

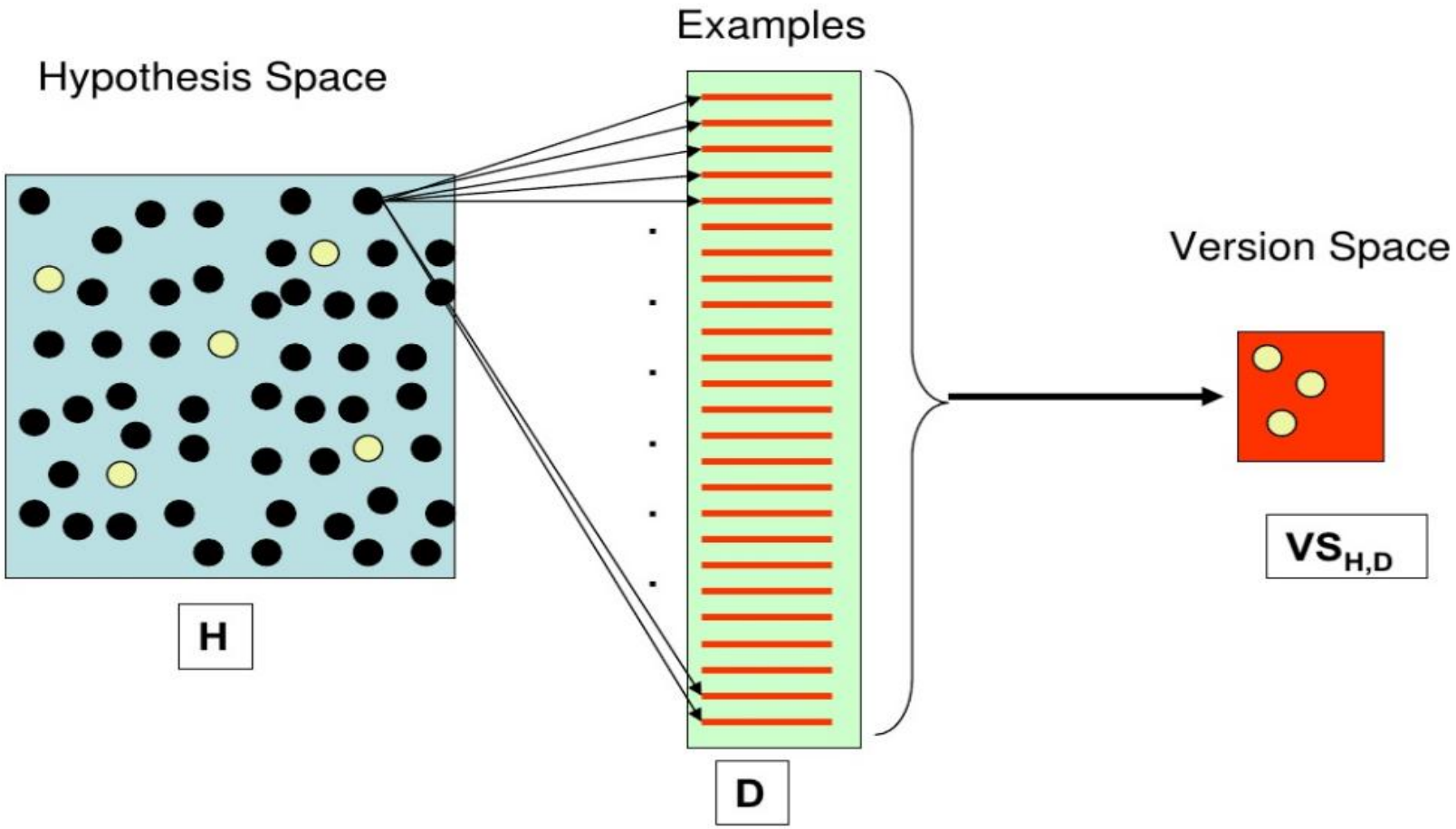
LIST-THEN-ELIMINATE Algorithm to Obtain Version Space

The LIST-THEN-ELIMINATE Algorithm

1. $VersionSpace \leftarrow$ a list containing every hypothesis in H
2. For each training example, $\langle x, c(x) \rangle$

   remove from $VersionSpace$ any hypothesis h for which $h(x) \neq c(x)$
3. Output the list of hypotheses in $VersionSpace$

LIST-THEN-ELIMINATE Algorithm to Obtain Version Space



LIST-THEN-ELIMINATE Algorithm to Obtain Version Space

- In principle, the **LIST-THEN-ELIMINATE** algorithm can be applied whenever the hypothesis space H is finite.

- It is guaranteed to output all hypotheses consistent with the training data.

- Unfortunately, it requires exhaustively enumerating all hypotheses in H-an unrealistic requirement for all but the most trivial hypothesis spaces.

2.More compact Representation

- Represented by most general and least general members
- They form general and specific boundary sets with partially ordered hypothesis space.

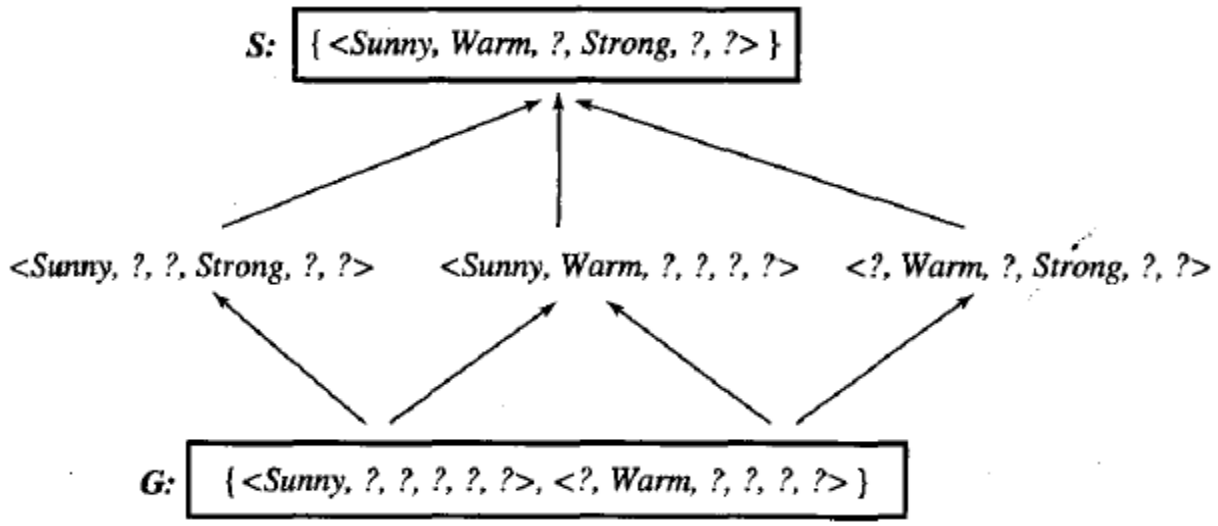


FIGURE 2.3
A version space with its general and specific boundary sets. The version space includes all six hypotheses shown here, but can be represented more simply by S and G. Arrows indicate instances of the *more_general_than* relation. This is the version space for the *EnjoySport* concept learning problem and training examples described in Table 2.1.

Boundary sets G and S

Definition: The **general boundary** G, with respect to hypothesis space H and training data D, is the set of maximally general members of H consistent with D.

$$G \equiv \{g \in H | Consistent(g, D) \wedge (\neg \exists g' \in H)[(g' >_g g) \wedge Consistent(g', D)]\}$$

Definition: The **specific boundary** S, with respect to hypothesis space H and training data D, is the set of minimally general (i.e., maximally specific) members of H consistent with D.

$$S \equiv \{s \in H | Consistent(s, D) \wedge (\neg \exists s' \in H)[(s >_g s') \wedge Consistent(s', D)]\}$$

Version space representation theorem

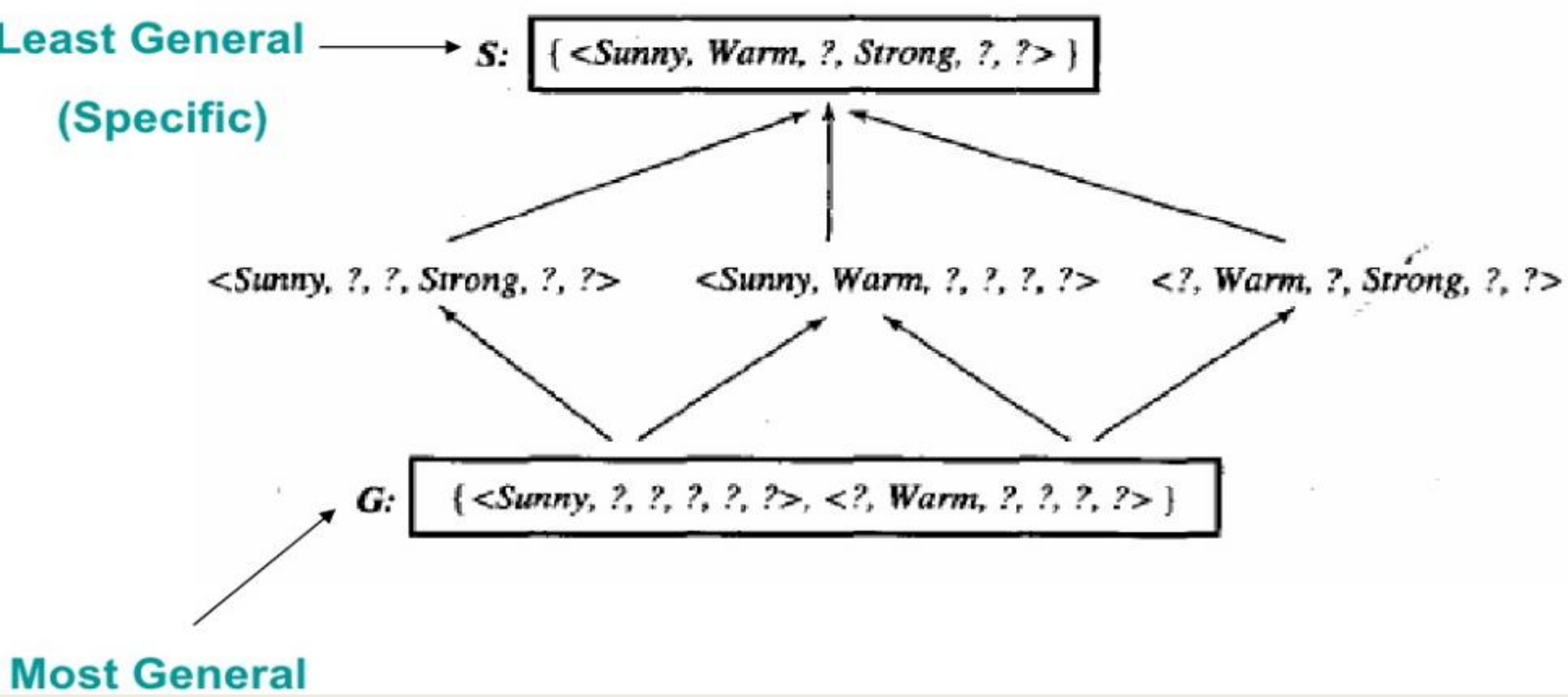
- Let X be any arbitrary set of instances
- Let H be set of Boolean values hypothesis defined over X
- Let c:X->[0,1] be arbitrary target defined over X
- Let D be an arbitrary set of training examples {<x, c(x)>} for all values of X,H,c,D such that S and G are well defined .

$$VS_{H,D} = \{h \in H | (\exists s \in S)(\exists g \in G)(g \geq_g h \geq_g s)\}$$

Candidate-Elimination Algorithm

- The **CANDIDATE-ELIMINATION** algorithm **works on the same principle** as the above **LIST-THEN-ELIMINATE** algorithm.
- It employs a much more **compact representation of the version space**.
- In this the **version space is represented by its most general and least general members (Specific)**.
- These members form general and specific boundary sets that delimit the version space within the partially ordered hypothesis space.

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Candidate-Elimination Algorithm

The **Candidate-Elimination** algorithm finds all describable hypotheses that are consistent with the observed training examples

Definition: A hypothesis h is **consistent** with a set of training examples D if and only if $h(x) = c(x)$ for each example $\langle x, c(x)\rangle$ in D.

$$Consistent(h, D) \equiv (\forall \langle x, c(x)\rangle \in D)\ h(x) = c(x)$$

Hypothesis is derived from examples regardless of whether x is positive or negative example

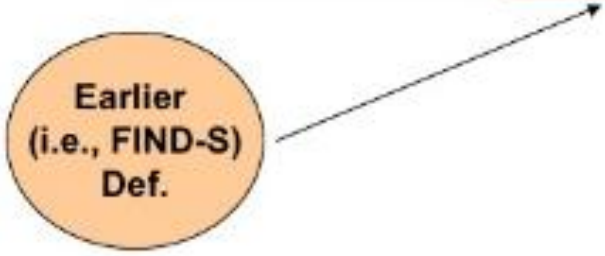
Candidate-Elimination Algorithm

Definition: A hypothesis h is **consistent** with a set of training examples D if and only if $h(x) = c(x)$ for each example $\langle x, c(x)\rangle$ in D.

$$Consistent(h, D) \equiv (\forall \langle x, c(x)\rangle \in D)\ h(x) = c(x)$$

Definition: Let h_j and h_k be **boolean-valued** functions defined over X. Then h_j is **more_general_than_or_equal_to** h_k (written $h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$



Candidate-Elimination Algorith

Initialize G to the set of maximally general hypotheses in H
Initialize S to the set of maximally specific hypotheses in H
For each training example d, do

- If d is a positive example
  - Remove from G any hypothesis inconsistent with d
  - For each hypothesis s in S that is not consistent with d
    - Remove s from S
    - Add to S all minimal generalizations h of s such that
      - h is consistent with d, and some member of G is more general than
    - Remove from S any hypothesis that is more general than another hypothes
- If d is a negative example
  - Remove from S any hypothesis inconsistent with d
  - For each hypothesis g in G that is not consistent with d
    - Remove g from G
    - Add to G all minimal specializations h of g such that
      - h is consistent with d, and some member of S is more specific than
    - Remove from G any hypothesis that is less general than another hypothesi

Example

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Initialization

$G0 \leftarrow \{<?, ?, ?, ?, ?, ?>\}$

$S0 \leftarrow \{<\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset>\}$

$G0 \leftarrow \{<?, ?, ?, ?, ?, ?>\}$

$S0 \leftarrow \{<\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset>\}$

Iteration 1

x1 = <Sunny, Warm, Normal, Strong, Warm, Same>

$G1 \leftarrow \{<?, ?, ?, ?, ?, ?>\}$

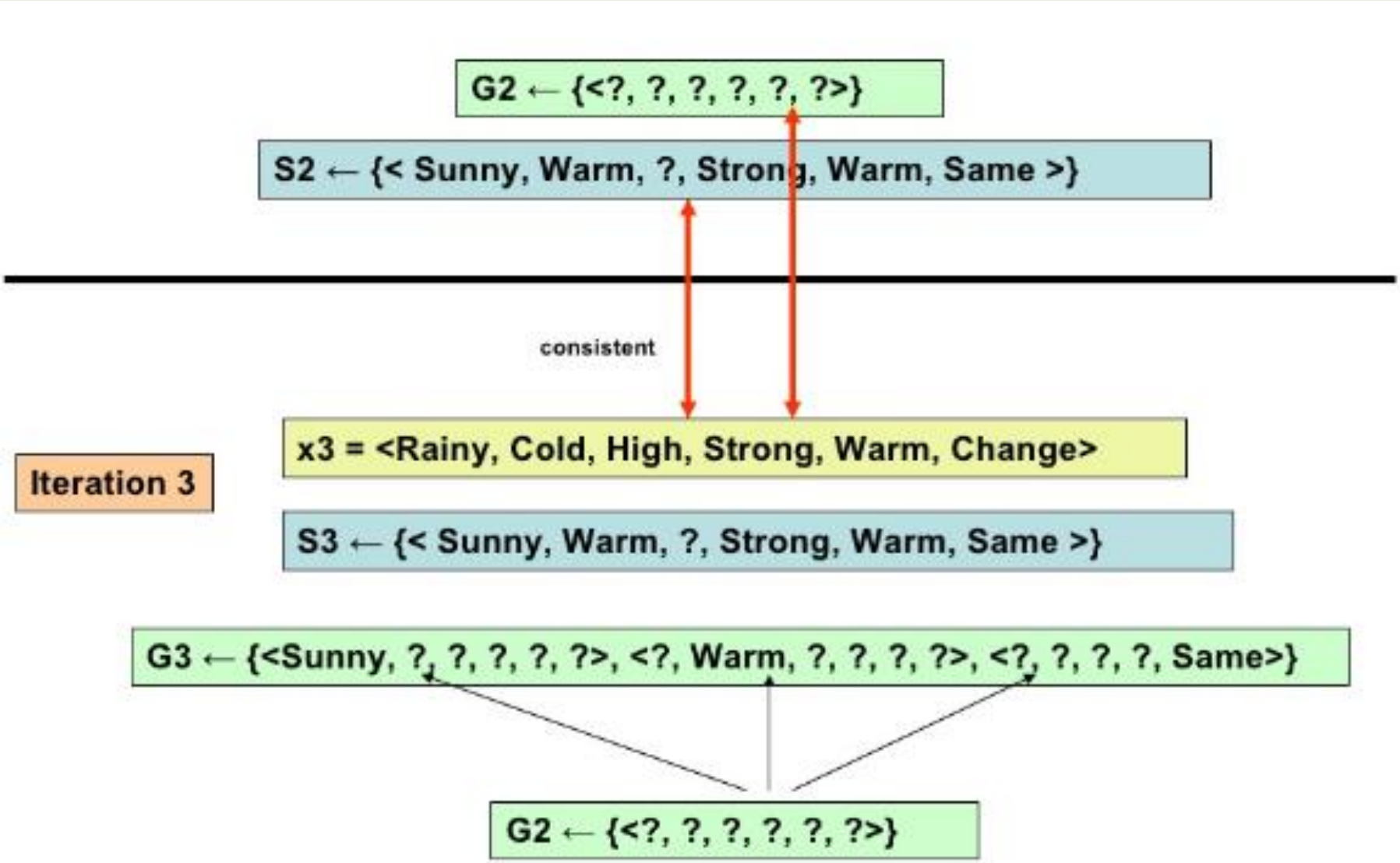
$S1 \leftarrow \{<$ Sunny, Warm, Normal, Strong, Warm, Same $>\}$

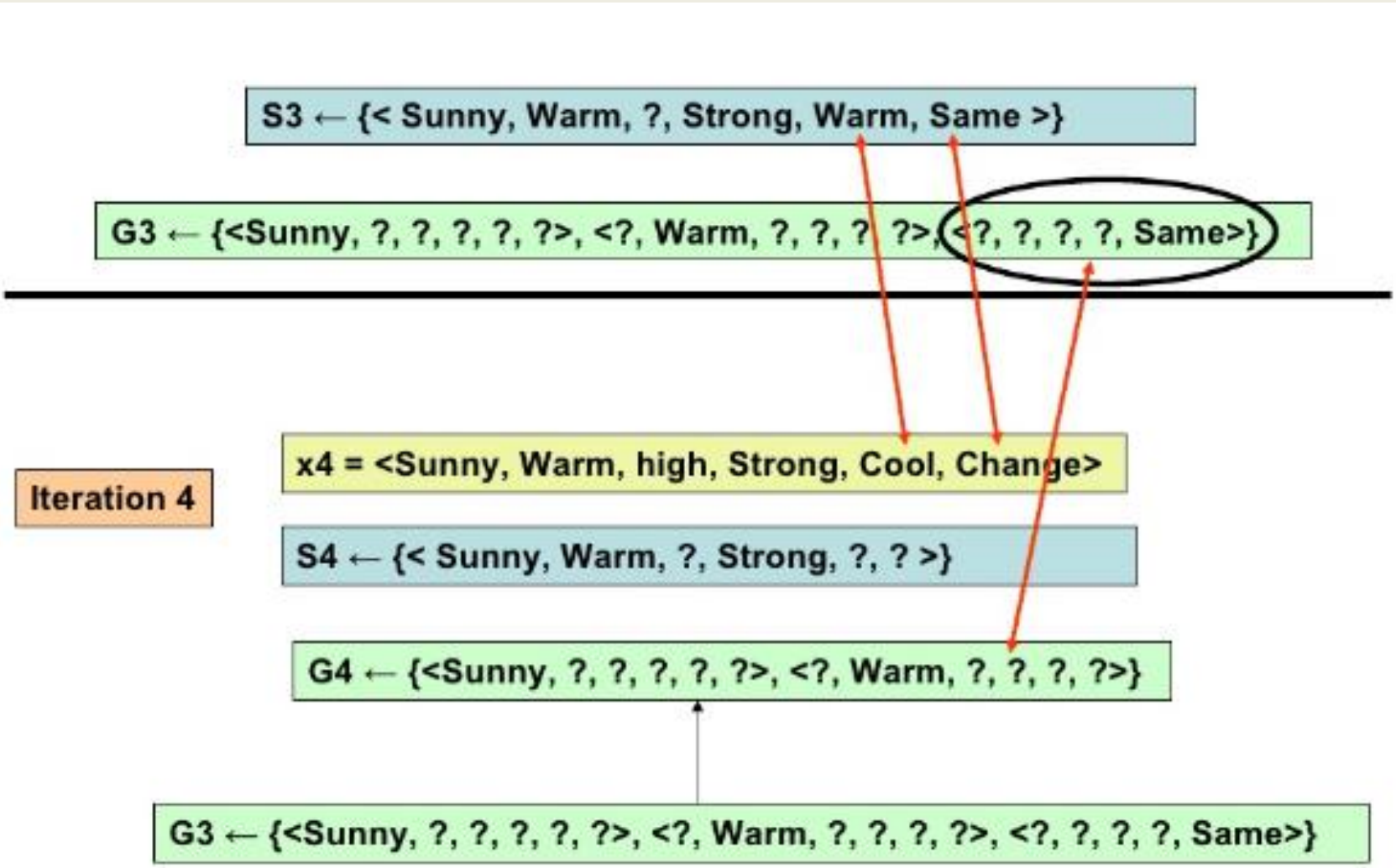
Iteration 2

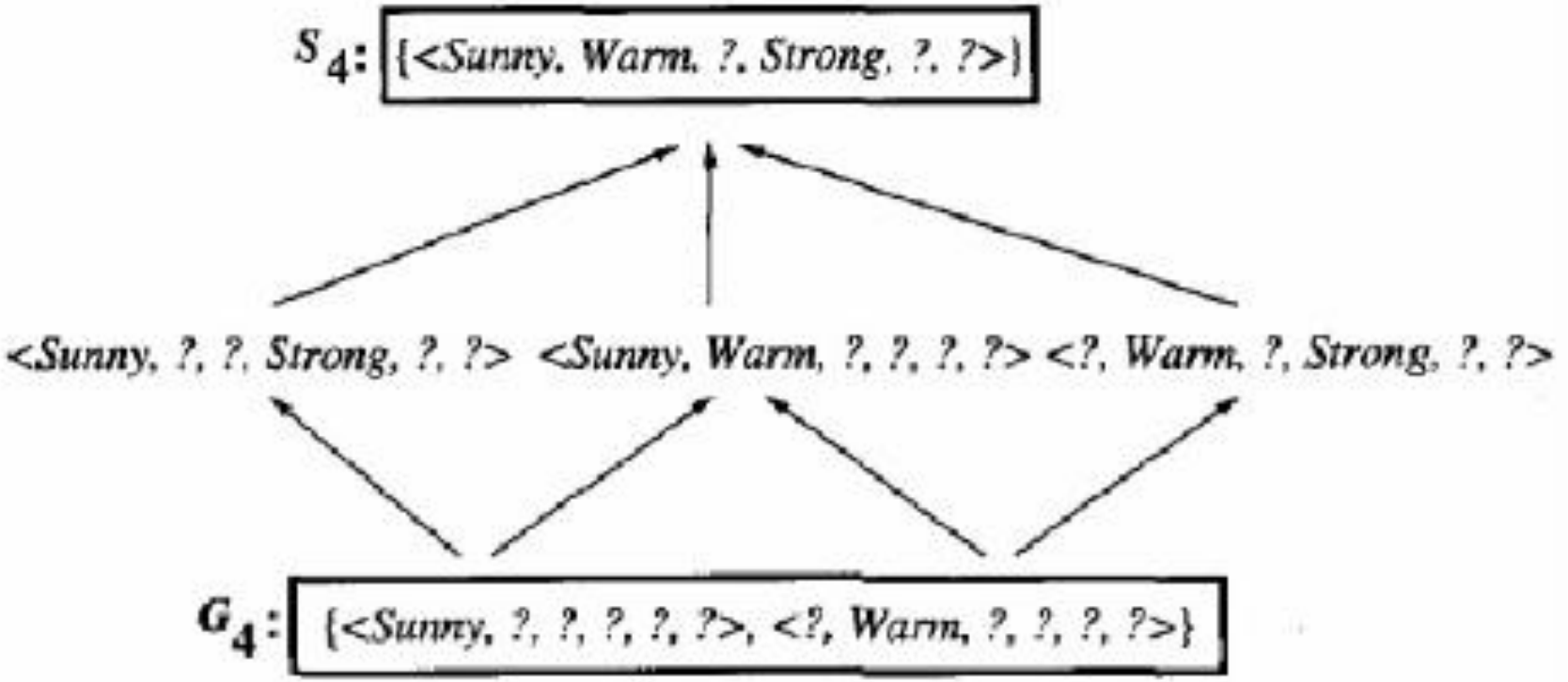
x2 = <Sunny, Warm, High, Strong, Warm, Same>

$G2 \leftarrow \{<?, ?, ?, ?, ?, ?>\}$

$S2 \leftarrow \{<$ Sunny, Warm, ?, Strong, Warm, Same $>\}$


G2 ← {<?, ?, ?, ?, ?, ?>}
S2 ← {< Sunny, Warm, ?, Strong, Warm, Same >}
consistent
x3 = <Rainy, Cold, High, Strong, Warm, Change>
Iteration 3
S3 ← {< Sunny, Warm, ?, Strong, Warm, Same >}
G3 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>, <?, ?, ?, ?, Same>}
G2 ← {<?, ?, ?, ?, ?, ?>}


S3 ← {< Sunny, Warm, ?, Strong, Warm, Same >}
G3 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>, <?, ?, ?, ?, Same>}
Iteration 4
x4 = <Sunny, Warm, high, Strong, Cool, Change>
S4 ← {< Sunny, Warm, ?, Strong, ?, ? >}
G4 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}
G3 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>, <?, ?, ?, ?, Same>}


S_4: {<Sunny, Warm, ?, Strong, ?, ?>}
<Sunny, ?, ?, Strong, ?, ?>
<Sunny, Warm, ?, ?, ?, ?>
<?, Warm, ?, Strong, ?, ?>
G_4: {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}

Example	*Sky*	*AirTemp*	*Humidity*	*Wind*	*Water*	*Forecast*	*EnjoySport*
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

<Sunny, Warm, ?, Strong, ?, ?>

<Sunny, ?, ?, Strong, ?, ?>

<Sunny, Warm, ?, ?, ?, ?>

<?, Warm, ?, Strong, ?, ?>

<Sunny, ?, ?, ?, ?, ?>

<?, Warm, ?, ?, ?, ?>

Remarks on Version Spaces and Candidate-Elimination

The version space learned by the **CANDIDATE-ELIMINATION** algorithm will converge toward the hypothesis that correctly describes the target concept, provided

(1) there are no errors in the training examples, and

(2) there is some hypothesis in H that correctly describes the target concept.

Perceptron

- Type of ANN is called as perceptron .

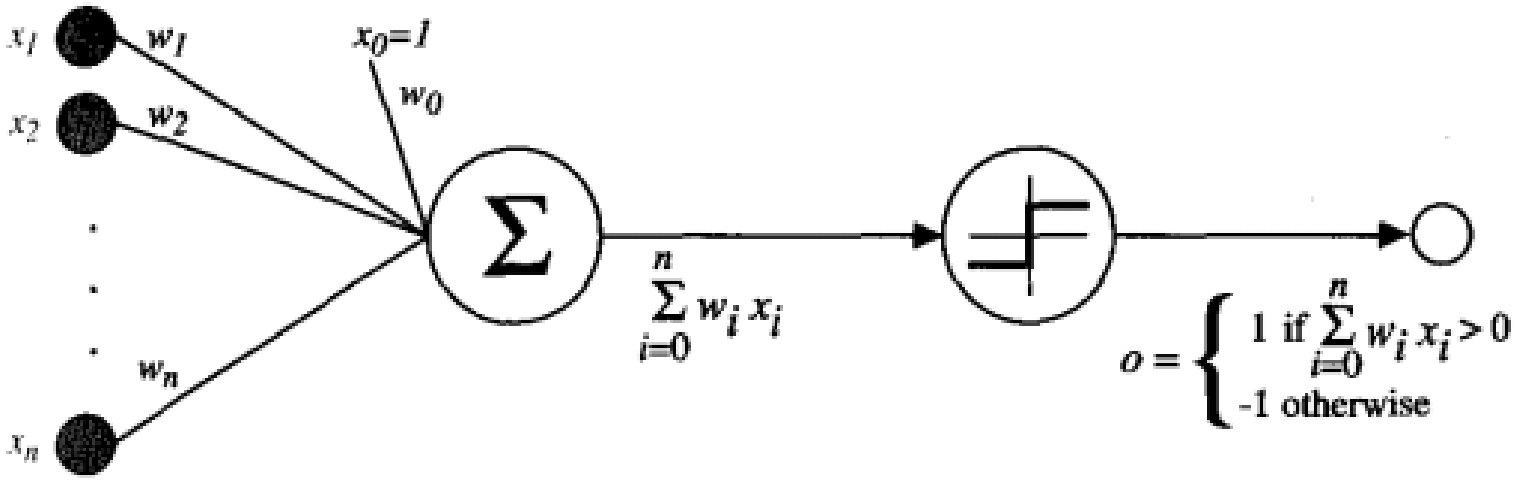


FIGURE 4.2
A perceptron.

- It takes a value of real valued inputs and calculates a linear combination of these inputs .
- Given <x1,x2, ….. , xn> as inputs and a (x1,x2,…. , xn) is computed by perceptron

$$o(x_1, \ldots, x_n) = \begin{cases} 1 \text{ if } w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n > 0 \\ -1 \text{ otherwise} \end{cases}$$

- W_i = real valued constant
- $-w_0$ is a threshold on weighted combinations of inputs .
- It gets the outputs as 1

$$w_1 x_1 + \cdots + w_n x_n$$

- If $x_0 = 1$, then $\sum_{i=0}^{n} w_i x_i > 0,$
- vector form is $\vec{w} \cdot \vec{x} > 0.$
- Perceptron function is

$$o(\vec{x}) = sgn(\vec{w} \cdot \vec{x})$$

where

$$sgn(y) = \begin{cases} 1 \text{ if } y > 0 \\ -1 \text{ otherwise} \end{cases}$$

- The space H of the candidate hypothesis considered in perceptron learning = set of (all possible real valued weight vectors)
- $$H = \{\vec{w} \mid \vec{w} \in \Re^{(n+1)}\}$$

Linearly separable

- +ve and –ve examples are separated by a hyperplane and it is called as linearly separable .
- If we assume boolean values of 1(true) and 0 (false)
- Perceptron can be represented as

AND, OR, NAND (¬ AND), and NOR (¬ OR).

XOR function

- Cannot be represented as its value is 1

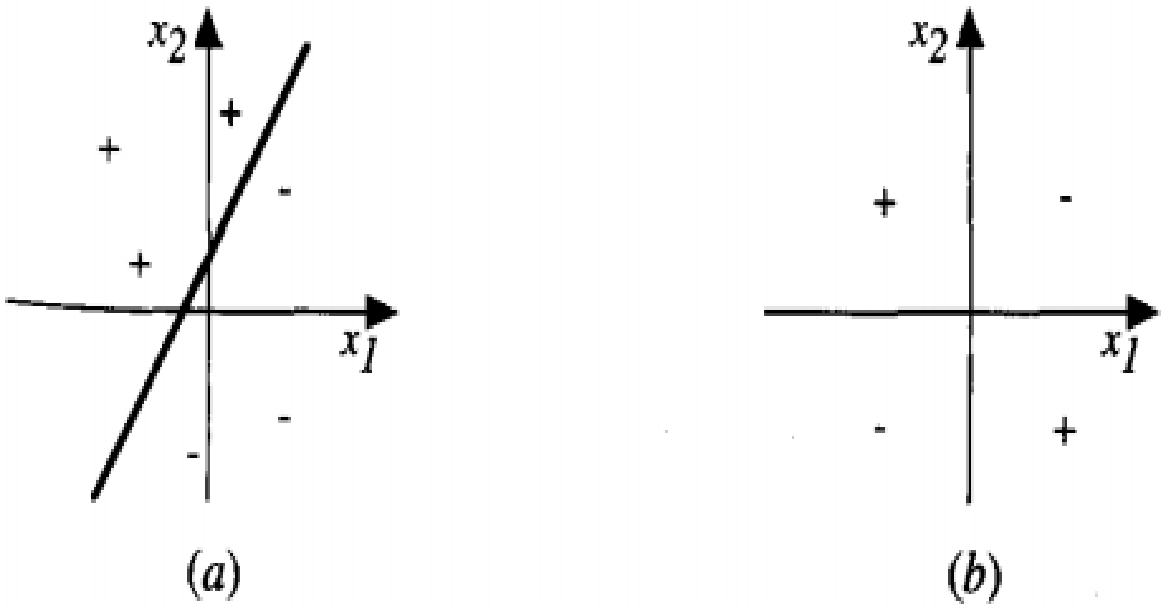


FIGURE 4.3

The decision surface represented by a two-input perceptron. (*a*) A set of training examples and the decision surface of a perceptron that classifies them correctly. (*b*) A set of training examples that is not linearly separable (i.e., that cannot be correctly classified by any straight line). x_1 and x_2 are the perceptron inputs. Positive examples are indicated by "+", negative by "−".

Perceptron

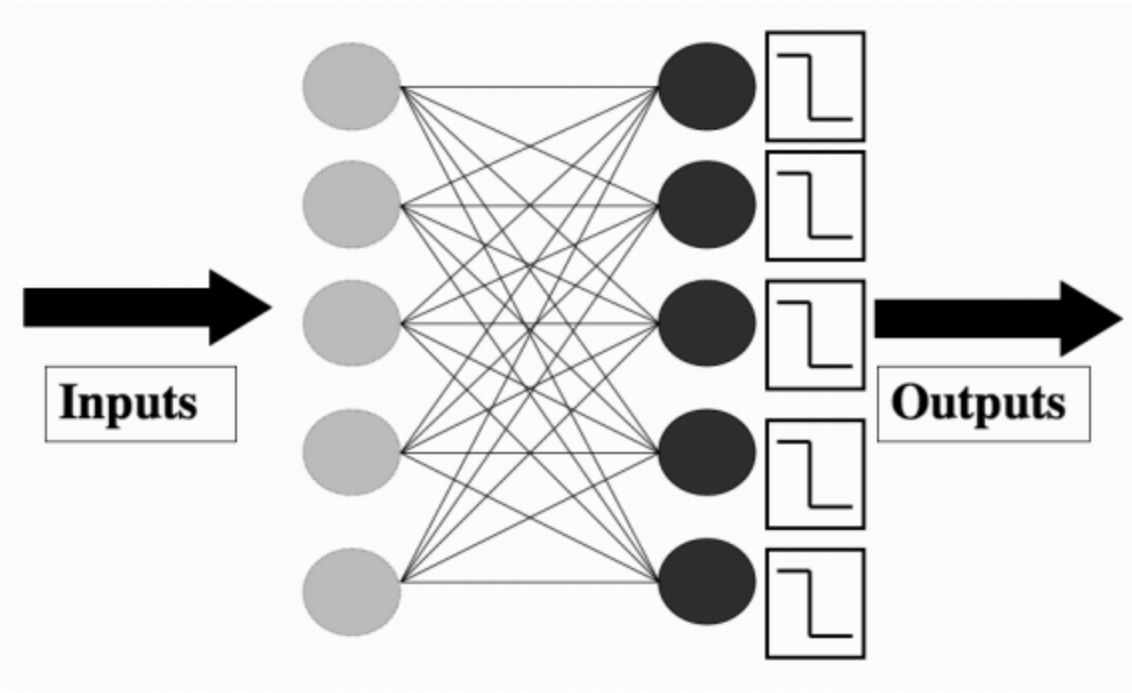


FIGURE 3.2 The Perceptron network, consisting of a set of input nodes (left) connected to McCulloch and Pitts neurons using weighted connections.

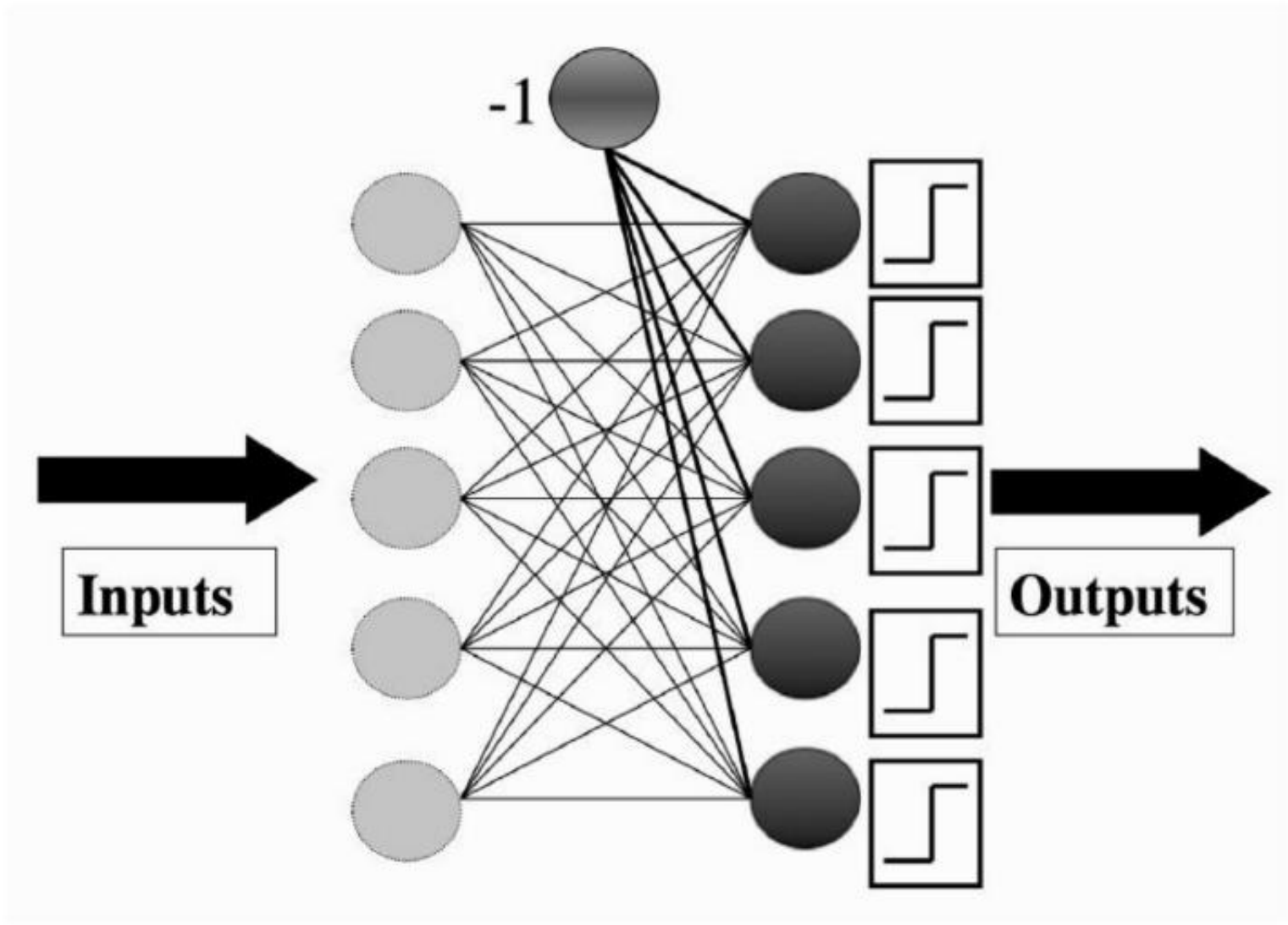


FIGURE 3.3 The Perceptron network again, showing the bias input.

- Learning rate is : $0.1 < \eta < 0.4$
- Bias: fix the value of the threshold for the neuron at zero. The input to the weight is ±1.
- The weight is -1 when the inputs are al;l zeros .This is called bias(w_{0j}) .

The Perceptron Algorithm

- **Initialisation**
  - set all of the weights w_{ij} to small (positive and negative) random numbers
- **Training**
  - for T iterations or until all the outputs are correct:
    - for each input vector:
      - compute the activation of each neuron j using activation function g:

$$y_j = g\left(\sum_{i=0}^{m} w_{ij}x_i\right) = \begin{cases} 1 & \text{if } \sum_{i=0}^{m} w_{ij}x_i > 0 \\ 0 & \text{if } \sum_{i=0}^{m} w_{ij}x_i \leq 0 \end{cases} \tag{3.4}$$

      - update each of the weights individually using:

$$w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j) \cdot x_i \tag{3.5}$$

- **Recall**
  - compute the activation of each neuron j using:

$$y_j = g\left(\sum_{i=0}^{m} w_{ij}x_i\right) = \begin{cases} 1 & \text{if } w_{ij}x_i > 0 \\ 0 & \text{if } w_{ij}x_i \leq 0 \end{cases} \tag{3.6}$$

In_1	In_2	t
0	0	0
0	1	1
1	0	1
1	1	1

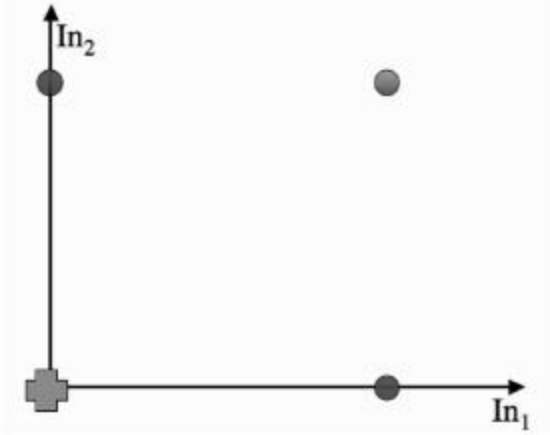


FIGURE 3.4 Data for the OR logic function and a plot of the four datapoints.

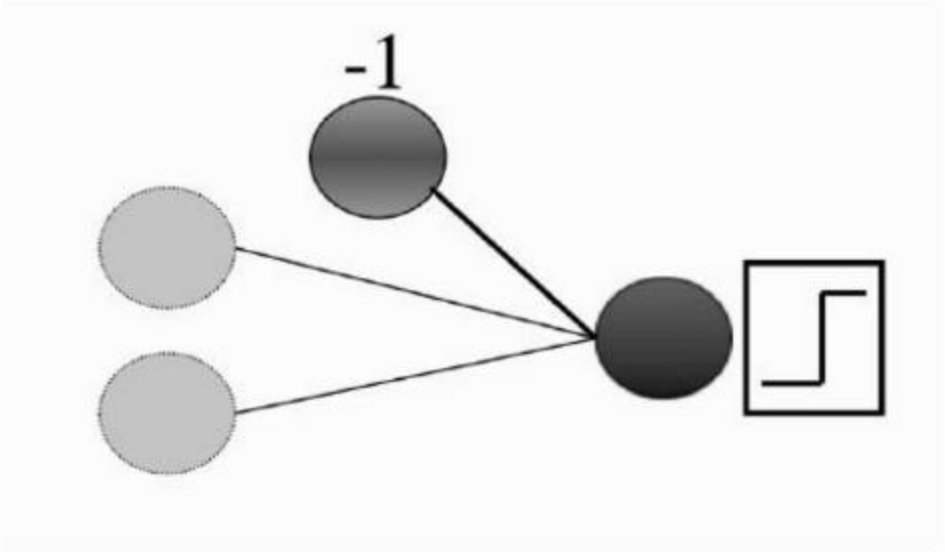


FIGURE 3.5 The Perceptron network for the example in Section 3.3.4.

complexity is O(mn).
For T iterations, costs is O(Tmn).

example

- w0 = −0.05, w1 = −0.02, w2 = 0.02.
- inputs are 0: (0, 0)
- Input to the bias weight is always −1,\
- value that reaches the neuron is −0.05 × −1 +−0.02 × 0 + 0.02 × 0 = 0.05

Result : This value is above 0, so the neuron fires and the output is 1, which is incorrect according to the target.

Apply the equation : $w_{ij} \leftarrow w_{ij} - \eta(y_j - t_j) \cdot x_i$ for $\eta = 0.25$

w0 : −0.05 − 0.25 × (1 − 0) × −1 = 0.2, (3.7)

w1 : −0.02 − 0.25 × (1 − 0) × 0 = −0.02, (3.8)

w2 : 0.02 − 0.25 × (1 − 0) × 0 = 0.02. (3.9)

feed in the next input (0, 1)

apply the learning rule again:

w0 : 0.2 − 0.25 × (0 − 1) × −1 = −0.05, (3.10)

w1 : −0.02 − 0.25 × (0 − 1) × 0 = −0.02, (3.11)

w2 : 0.02 − 0.25 × (0 − 1) × 1 = 0.27. (3.12)

(1, 0) input the answer is already correct

.

```
for data in range(nData): # loop over the input vectors
    for n in range(N): # loop over the neurons
        # Compute sum of weights times inputs for each neuron
        # Set the activation to 0 to start
        activation[data][n] = 0
        # Loop over the input nodes (+1 for the bias node)
        for m in range(M+1):
            activation[data][n] += weight[m][n] * inputs[data][m]

        # Now decide whether the neuron fires or not
        if activation[data][n] > 0:
            activation[data][n] = 1
        else
            activation[data][n] = 0
```

Linear Regression

- Linear regression is a basic and commonly used type of predictive analysis.
- The overall idea of regression is to examine two things:

- (1) does a set of predictor variables do a good job in predicting an outcome (dependent) variable?

- (2) Which variables in particular are significant predictors of the outcome variable, and in what way do they–indicated by the magnitude and sign of the beta estimates–impact the outcome variable?

- These regression estimates are used to explain the relationship between one dependent variable and one or more independent variables.

- The simplest form of the regression equation with one dependent and one independent variable is defined by the formula

$$y = c + b*x$$

where y = estimated dependent variable score,
c = constant,
b = regression coefficient,
x = score on the independent variable.

- Naming the Variables. There are many names for a regression's dependent variable. It may be called an outcome variable, criterion variable, endogenous variable, or regressand.

- The independent variables can be called exogenous variables, predictor variables, or regressors.

- Three major uses for regression analysis are

(1)determining the strength of predictors,
(2)forecasting an effect, and
(3) trend forecasting.

- First, the regression might be used to identify the strength of the effect that the independent variable(s) have on a dependent variable. Typical questions are what is the strength of relationship between dose and effect, sales and marketing spending, or age and income.

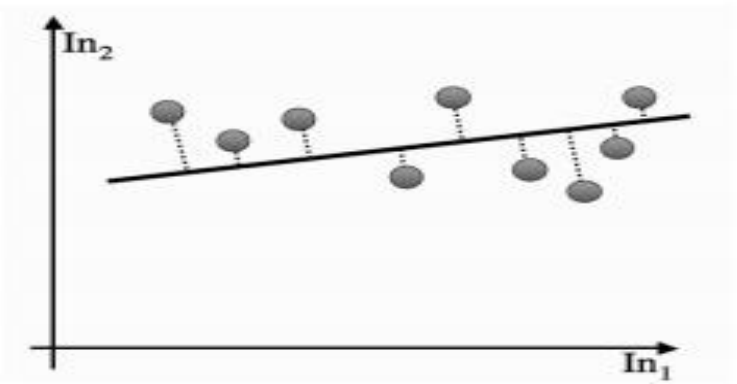
- Second, it can be used to forecast effects or impact of changes. That is, the regression analysis helps us to understand how much the dependent variable changes with a change in one or more independent variables.
- A typical question is, "how much additional sales income do I get for each additional $1000 spent on marketing?"

- Third, regression analysis predicts trends and future values. The regression analysis can be used to get point estimates. A typical question is, "what will the price of gold be in 6 months?"

Linear Regression

As is common in statistics, we need to separate out regression problems, where we fit a line to data, from classification problems, where we find a line that separates out the classes, so that they can be distinguished. However, it is common to turn classification problems into regression problems. This can be done in two ways, first by introducing an indicator variable, which simply says which class each datapoint belongs to. The problem is now to use the data to predict the indicator variable, which is a regression problem. The second approach is to do repeated regression, once for each class, with the indicator value being 1 for examples in the class and 0 for all of the others. Since classification can be replaced by regression using these methods, we'll think about regression here.

The only real difference between the Perceptron and more statistical approaches is in the way that the problem is set up. For regression we are making a prediction about an unknown value y (such as the indicator variable for classes or a future value of some data) by computing some function of known values x_i. We are thinking about straight lines, so the output y is going to be a sum of the x_i values, each multiplied by a constant parameter: $y = \sum_{i=0}^{M} \beta_i x_i$. The β_i define a straight line (plane in 3D, hyperplane in higher dimensions) that goes through (or at least near) the datapoints. Figure 3.13 shows this in two and three dimensions.



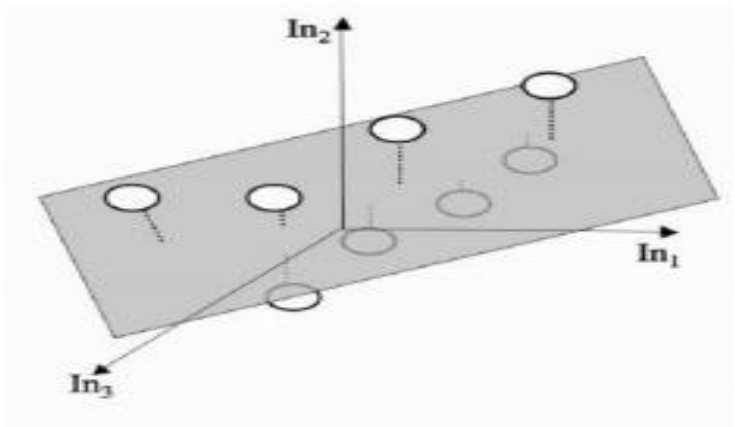


FIGURE 3.13 Linear regression in two and three dimensions.

- Linear Regression

we can use Pythagoras' theorem to know the distance. Now, we can try to minimise an error function that measures the sum of all these distances. If we ignore the square roots, and just minimise the sum-of-squares of the errors, then we get the most common minimisation, which is known as least-squares optimisation. What we are doing is choosing the parameters in order to minimise the squared difference between the prediction and the actual data value, summed over all of the datapoints. That is, we have:

$$\sum_{j=0}^{N} \left(t_j - \sum_{i=0}^{M} \beta_i x_{ij} \right)^2 . \tag{3.21}$$

This can be written in matrix form as:

$$(\mathbf{t} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{t} - \mathbf{X}\boldsymbol{\beta}), \tag{3.22}$$

where $\mathbf{t}$ is a column vector containing the targets and $\mathbf{X}$ is the matrix of input values (even including the bias inputs), just as for the Perceptron. Computing the smallest value of this means differentiating it with respect to the (column) parameter vector $\boldsymbol{\beta}$ and setting the derivative to 0, which means that $\mathbf{X}^T(\mathbf{t} - \mathbf{X}\boldsymbol{\beta}) = 0$ (to see this, expand out the brackets, remembering that $\mathbf{A}\mathbf{B}^T = \mathbf{B}^T\mathbf{A}$ and note that the term $\boldsymbol{\beta}^T\mathbf{X}^t\mathbf{t} = \mathbf{t}^T\mathbf{X}\boldsymbol{\beta}$ since they are

both a scalar term), which has the solution $\beta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{t}$ (assuming that the matrix $\mathbf{X}^T\mathbf{X}$ can be inverted). Now, for a given input vector $\mathbf{z}$, the prediction is $\mathbf{z}\beta$. The inverse of a matrix $\mathbf{X}$ is the matrix that satisfies $\mathbf{X}\mathbf{X}^{-1} = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix, the matrix that has 1s on the leading diagonal and 0s everywhere else. The inverse of a matrix only exists if the matrix is square (has the same number of rows as columns) and its determinant is non-zero.